IMPERIAL

# Neural Language Models and Sequence Modelling

Shamsuddeen Muhammad
Google DeepMind Academic Fellow,
Imperial College London
https://shmuhammadd.github.io/

# Today

**Chapter 6 Neural Networks:**

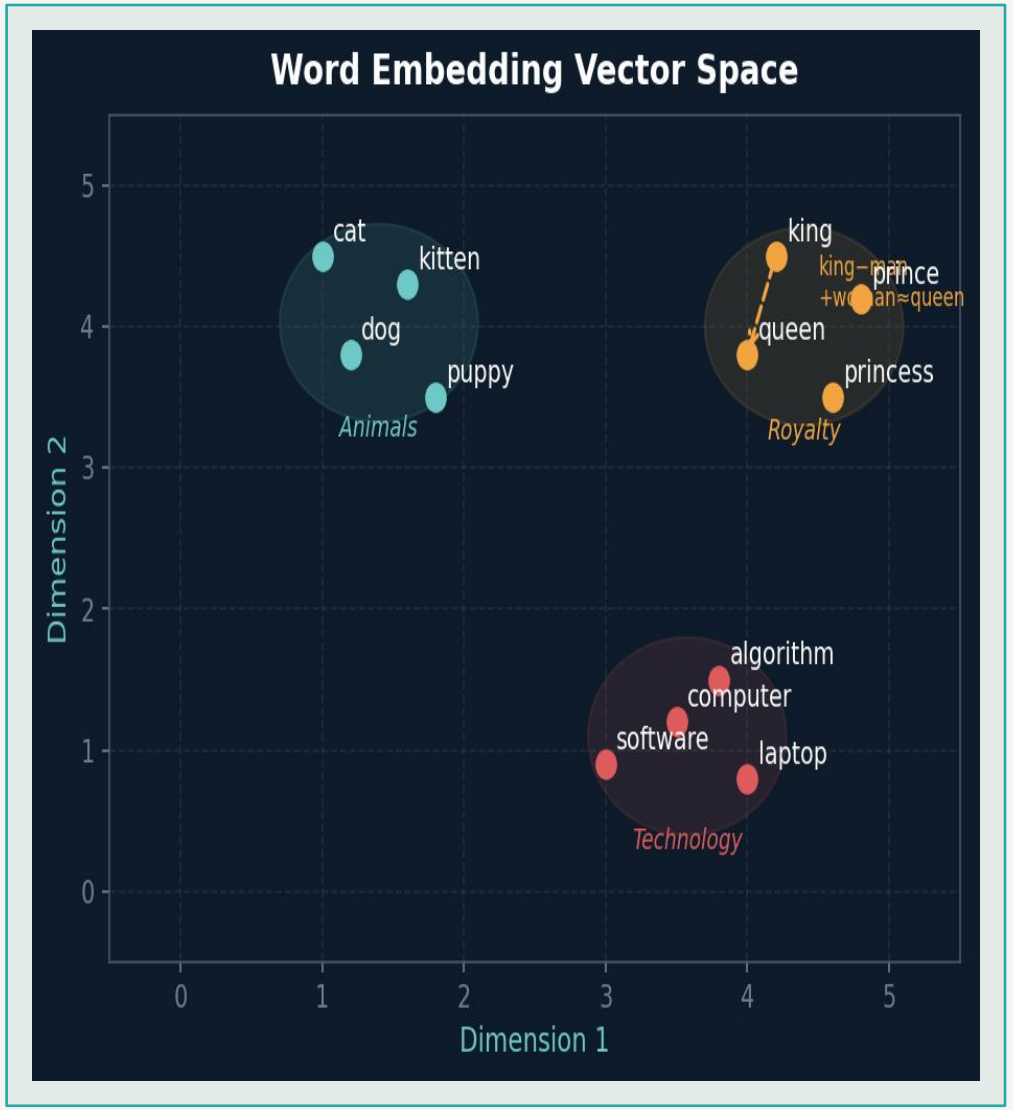
https://web.stanford.edu/~jurafsky/slp3/6.pdf

# Prerequisite

- Prerequisite
  - Neural Network
  - Loss function, backpropagation
  - CNN

# Last Lecture: The Vector Space Hypothesis

- Words with similar meaning occupy nearby regions in vector space
- One-Hot Encoding: each word = isolated dimension, no semantic relation
  - *→ 'dog' and 'puppy' are just as distant as 'dog' and 'democracy'*
- Distributional Semantics: words are defined by their contexts (Harris, 1954)
- Word2Vec: CBOW & Skip-gram — learn dense embeddings by predicting context
- GloVe: Global Vectors — co-occurrence statistics + local context combined
- FastText: sub-word embeddings — handles morphology and OOV words



# Word Embeddings: What We Learned

## Key Properties

- Dense, low-dimensional vectors
- king − man + woman ≈ queen (analogy arithmetic)
- Cosine similarity ≡ semantic similarity
- **Transfer learning**: train once, reuse anywhere

## Open Questions

- A word has ONE static vector — what about polysemy?
- 'Bank' (river) vs 'Bank' (financial) — same embedding!
- Embeddings ignore word ORDER in a sentence
- How do we model full sentences and meaning?
- This is the bridge to today's lecture →

*ELMo (2018) partially solves polysemy — Transformers solve it completely*

# The Critical Gap: From Words to Sequences

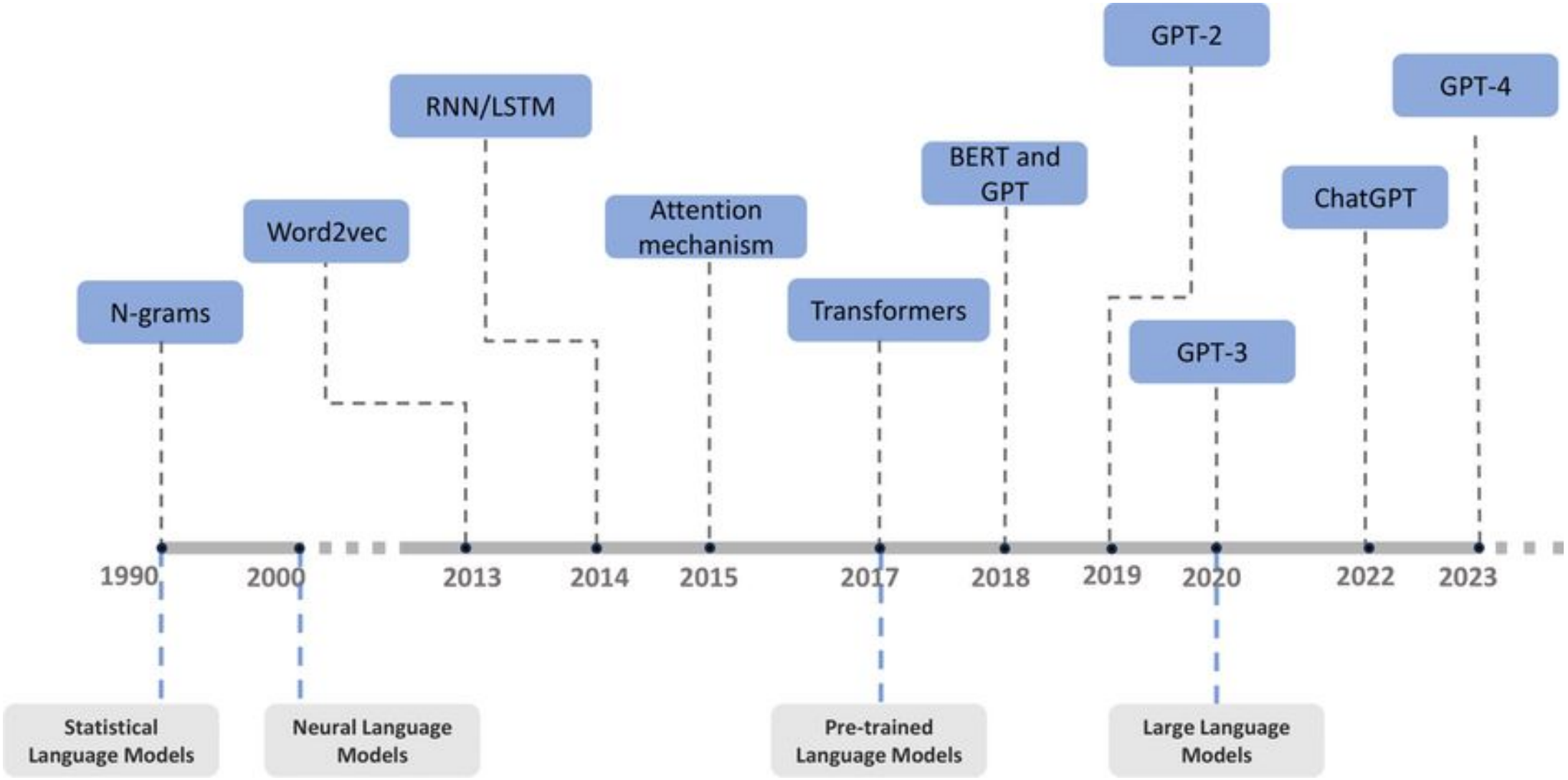
- **Problem: language is inherently sequential and compositional**
  - Word vectors give us building blocks — but sentences are not bags of words
  - "The dog bit the man" ≠ "The man bit the dog" — same words, opposite meaning
  - We need models that respect ORDER and CONTEXT across the entire sequence
  - **Today: neural architectures that process sequences — not just individual tokens**
    - Static embeddings → CNN → RNN → LSTM → Attention → Transformer

“

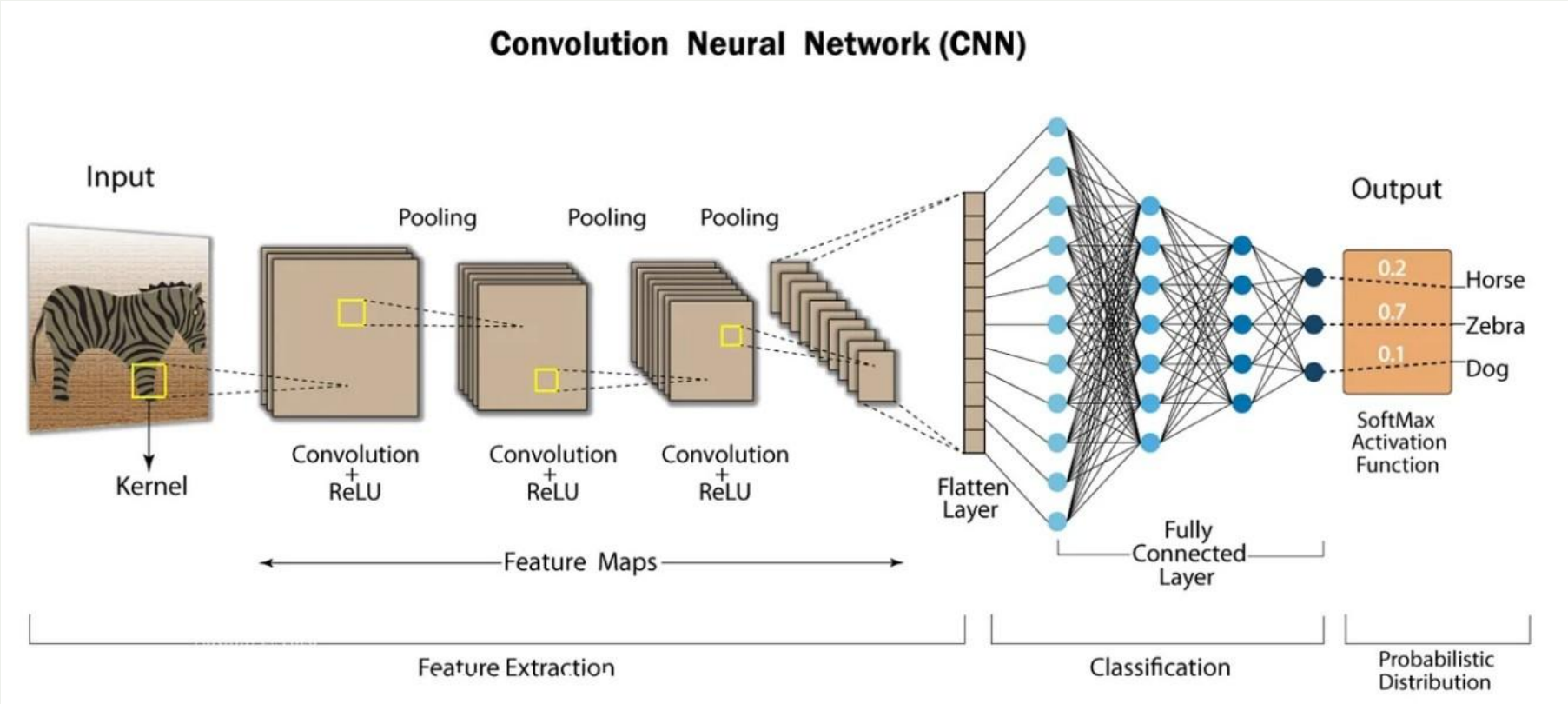
*Language is not merely a bag of words but a tool with particular properties.*

— John R. Firth, 1957

# From N-grams to Large Language Models

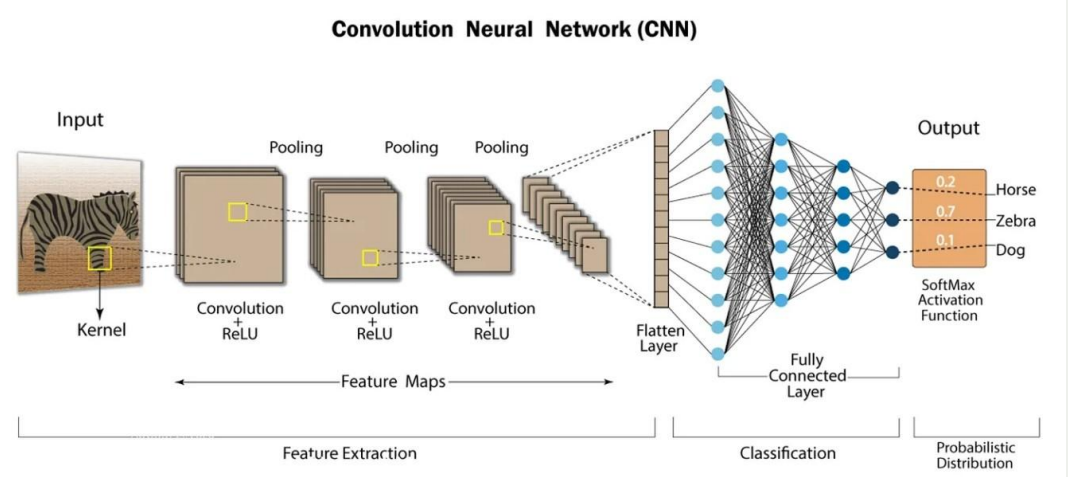


# CNN Revisited

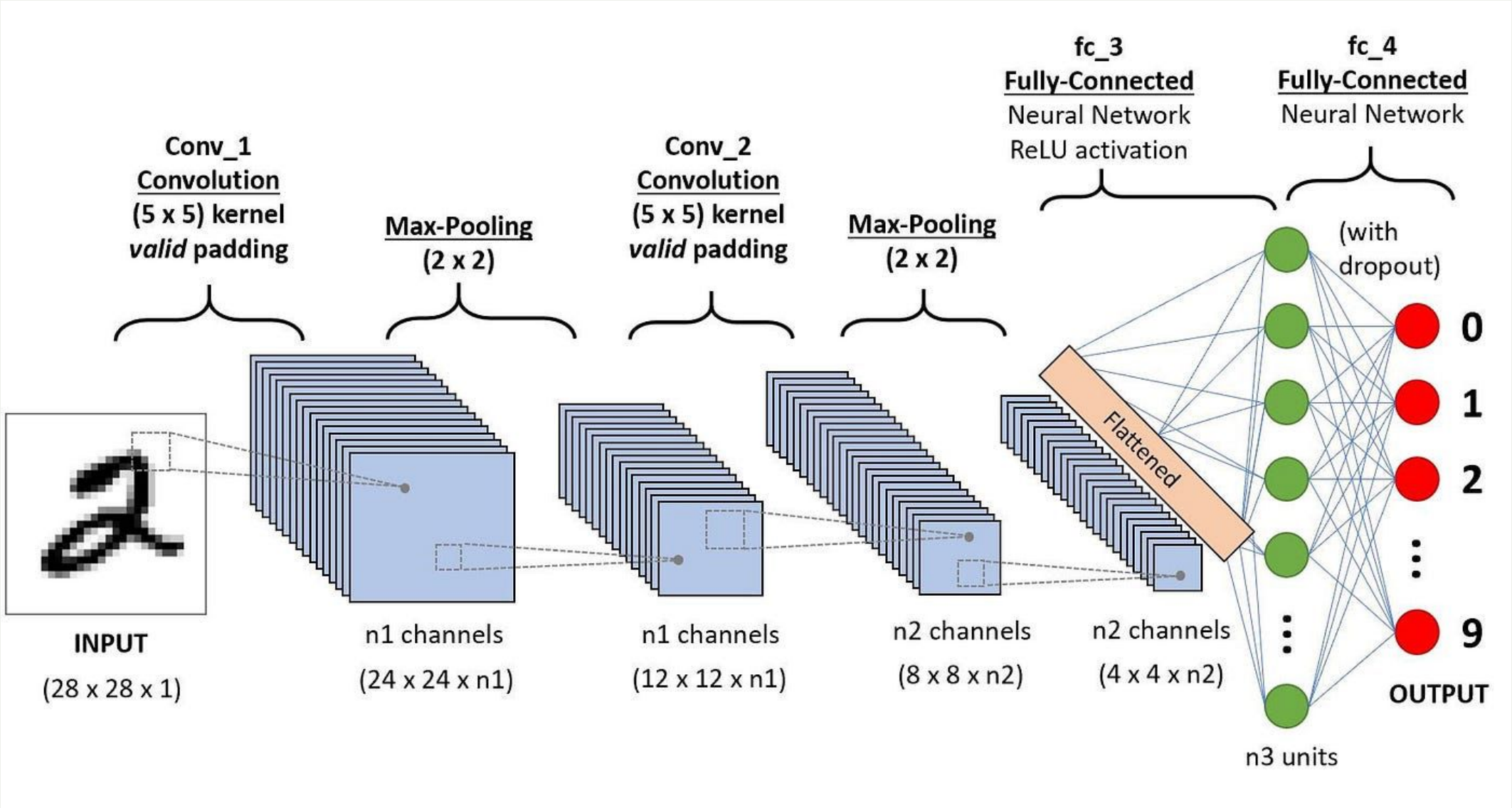



- A **CNN** is a type of neural network that's designed to work with ***image data***.
- Robust Performance: Achieves state-of-the-art results in computer vision tasks.

# CNN Revisited



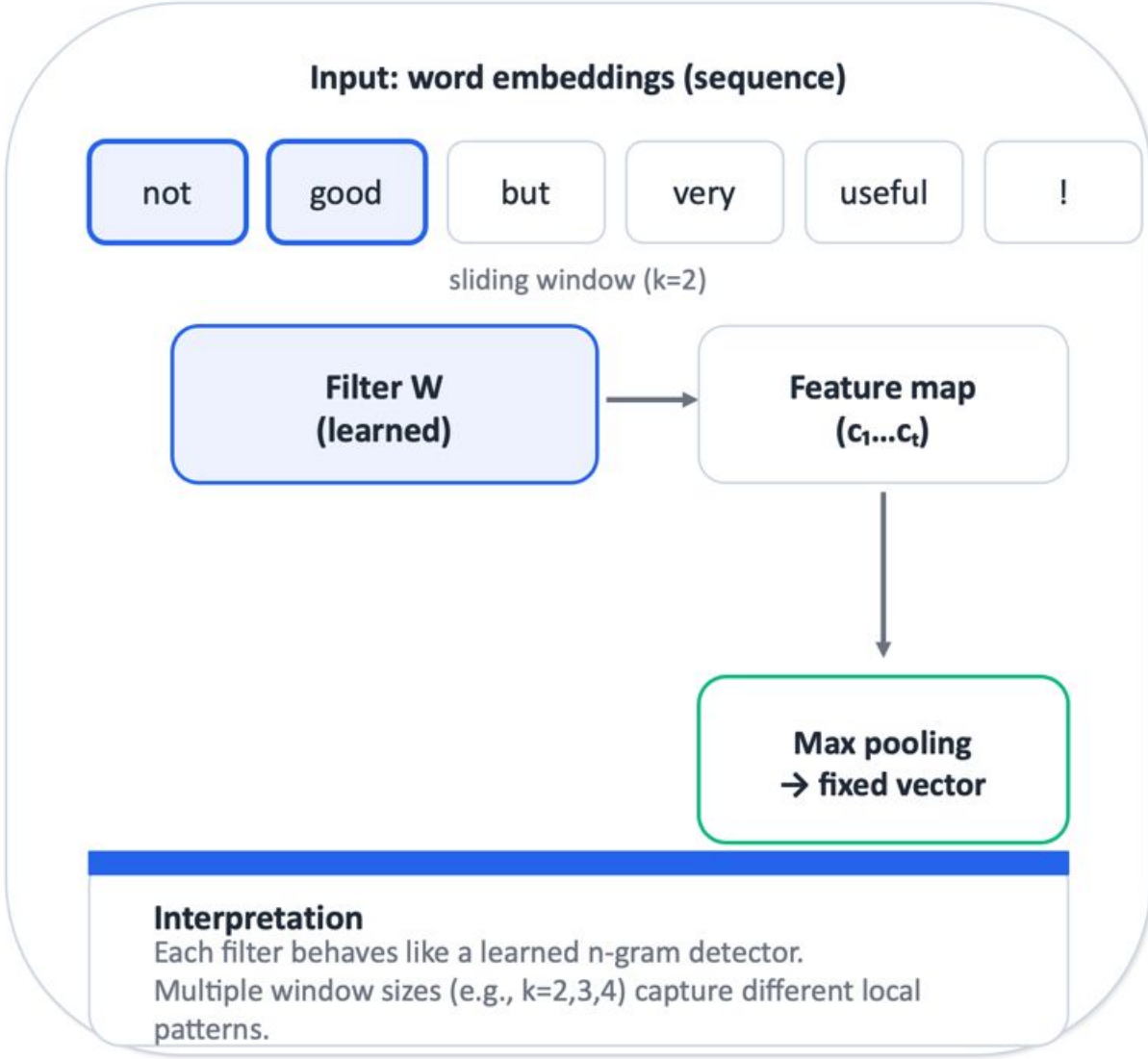

- They are made up of a series of layers:
  - The **convolutional** layers: responsible for extracting features from the images.
  - The **pooling** layers: responsible for reducing the size of the image after the convolutional layers have extracted the features.
  - **Activation** layers: Introduce non-linearity to the model (ReLU (Rectified Linear Unit), Sigmoid)


Conv_1
Convolution
(5 x 5) kernel
valid padding
Max-Pooling
(2 x 2)
Conv_2
Convolution
(5 x 5) kernel
valid padding
Max-Pooling
(2 x 2)
fc_3
Fully-Connected
Neural Network
ReLU activation
fc_4
Fully-Connected
Neural Network
(with dropout)
Flattened
INPUT
(28 x 28 x 1)
n1 channels
(24 x 24 x n1)
n1 channels
(12 x 12 x n1)
n2 channels
(8 x 8 x n2)
n2 channels
(4 x 4 x n2)
n3 units
0
1
2
9
OUTPUT

While CNNs were originally designed for images (2D), they are highly effective for text because they can identify n $n$ -gram features, such as key phrases or sentiment-carrying word combinations—regardless of where they appear in a sentence

A 1D Convolutional Neural Network (CNN) is used for tasks where the input data is one-dimensional, such as sequences of words, time series data, or signals.

# CNNs for Text: Local Pattern Extraction




- Input is a sequence of embeddings ($T \times d$).
- A filter of width k scores each k-gram span via a dot product + nonlinearity.
- Many filters learn diverse local cues (negation, intensifiers, domain phrases).
- Pooling yields a fixed-size sentence vector for classification.

# Why Plain CNNs Can Be a Poor Fit for Many NLP Tasks

**Key limitations (plain CNN + pooling)**

Local receptive field:
• Direct interactions are limited to k-token windows.
• Long-range dependencies require many layers, dilation, or extra mechanisms.

Order information can be weakened:
• Global max pooling keeps “what” matched, but not “where”.

Token-level predictions are awkward:
• Tasks like tagging (NER/POS) often need per-token, context-rich states.

Harder for generative / autoregressive modeling:
• Many NLP problems naturally condition on a growing prefix.

**Where CNNs are strong**

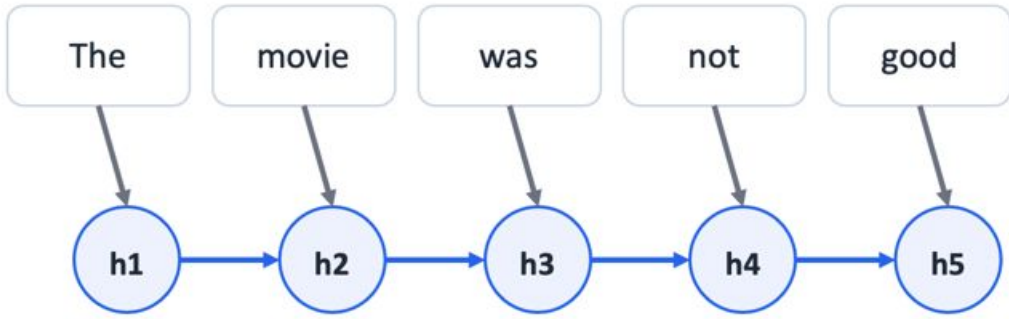
- • Sentence/document classification
- • Fast & parallel feature extraction
- • Robust local pattern detection

**Takeaway**

CNNs are not “bad” for NLP—
plain CNNs often struggle when the task needs
(1) long-distance interactions,
(2) strong word-order sensitivity, or
(3) rich per-token context.

# Classical Sequence Architectures: How They Address the Gaps

### RNN/LSTM/GRU: hidden state carries context




Forward recurrence (captures prefix context)

**Bidirectional RNN (BiLSTM/BiGRU)**

Reads left→right and right→left to build representations with both past and future context (useful for tagging and extraction).

**Seq2Seq + Attention (classical bridge)**

Rather than compressing the whole source into one vector, attention lets the model "look back" at relevant tokens when predicting outputs—helpful for longer sentences.

### How sequence models solve CNN pain points

Long-range dependencies:

- Hidden state (and gates in LSTM/GRU) can carry information across many steps.

Order sensitivity:

- Representations are computed sequentially; position is implicit in the computation.

Per-token outputs:

- Natural fit for sequence labeling (NER/POS), ASR alignment, etc.

Attention as an explicit bridge:

- Reduces the "fixed-length bottleneck" in encoder–decoder models, especially on longer inputs.

Trade-off:

- Less parallelizable than CNNs (sequential dependency).

## Understanding 1D CNN

- Input Data: Unlike 2D CNNs, which process images with width and height, 1D CNNs operate on sequences with only one dimension. Examples include time series data, text (sequence of words), or any data where the ordering of elements matters.
- 1D Convolution Operation: The 1D convolutional layer slides a filter (kernel) over the sequence, computing the dot product between the filter and the input sequence at each position. This produces a feature map highlighting important patterns in the sequence.

# When to Use 1D CNNs

- Time Series Analysis: Detecting patterns in temporal data.
- Natural Language Processing (NLP): For tasks like text classification, where sequences of words (or characters) are processed.
- Signal Processing: Analyzing signals such as ECG, EEG, or audio data.

# CNN Revisited

In NLP, CNNs apply **1-dimensional convolutions** over sequences of word embeddings to automatically learn **local linguistic patterns** (e.g., n-grams such as *"not good"*, *"very happy"*)

Kim, Y. (2014). *Convolutional Neural Networks for Sentence Classification.*

# CNN Revisited

- Lack of Sequential Order Awareness
  - CNNs treat input as spatially ordered (like pixels in an image) but not temporally ordered.
- In sequential tasks (e.g., text or time-series data), the order of input is critical.

# CNN Revisited

CNNs require the input data to have a **fixed size** (e.g., images of a specific resolution).

This makes them unsuitable for sequences of varying lengths.

# What is Sequence Modelling?

# Limitation of N-Gram Language Model

- N-gram models struggle to capture **long-range dependencies** in natural language.
- They only look at the previous *n* words, so important relationships across the sentence are lost.
- They lack global coherence and fail on complex syntax.

# Traditional Models

The following classical approach treats text as unordered.

- Rule-Based & Pattern Matching Systems
- Classical Machine Learning : Logistic Regression, Naive Bayes, SVM (Support Vector Machines), Decision Trees / Random Forests
- TF-IDF + Linear Classifiers (Logistic Regression, SVM)
- Topic Models such as LDA

# Limitations of Classical Models

- Ignore word order (bag-of-words assumption)
- Cannot capture long-range dependencies
- Fail on tasks needing context
- Limited semantic understanding

**Simple Example for Students**

Sentence A: *"Football makes people happy."*
Sentence B: *"People make football happy."*

Both contain the same words - BoW models treat them as identical, even though the meanings differ.

# Sequence Modelling

Natural language is inherently sequential: meaning depends on previous and next words.

Sequence modelling refers to building models that can ***represent, process, and predict ordered data***, *where* ***the order of elements matters***.

# Defining the Sequence Modelling Problem

- A sequence is an ordered list where position matters: $x_1$, $x_2$, …, $x_{\square}$
- **Core task:** learn a function that maps input sequences to outputs, respecting order
- Input and output may be sequences themselves (many-to-many)
- **Key property:** the model must handle VARIABLE LENGTH inputs

# Variable Length in Sequence Modelling — What Does It Mean?

**Definition**

Different inputs have different numbers of tokens — and the model must handle ALL of them with the SAME parameters.

**Examples**

| Sentence | Length |
|---|---|
| "Go!" | 1 token |
| "The cat sat on the mat." | 6 tokens |
| "The animal didn't cross the street…" | 11 tokens |
| A full paragraph | 100+ tokens |

⚠️ **The Problem with Fixed-Size Networks**

A standard FFNN has fixed input dimensions. Design it for 10 words → it cannot accept 5 or 50. Like a form with exactly 10 blank fields.

## How Each Architecture Handles Variable Length

| | |
|---|---|
| **N-grams** | Only look at last n words — ignore the rest.<br>✗ *Crude workaround — loses history* |
| **CNNs** | Max-pooling collapses variable length into fixed size.<br>✗ *Loses word order in the process* |
| **RNNs** | Process one token at a time — keep running until sequence ends.<br>✔ *Naturally handles any length* |
| **Transformers** | Attend over all tokens at once — any length (up to context window).<br>✔ *Elegant — full context, no recurrence* |

**RNNs were the first architecture to elegantly solve variable length — $h_t = f(x_t, h_{t-1})$ just keeps running.**

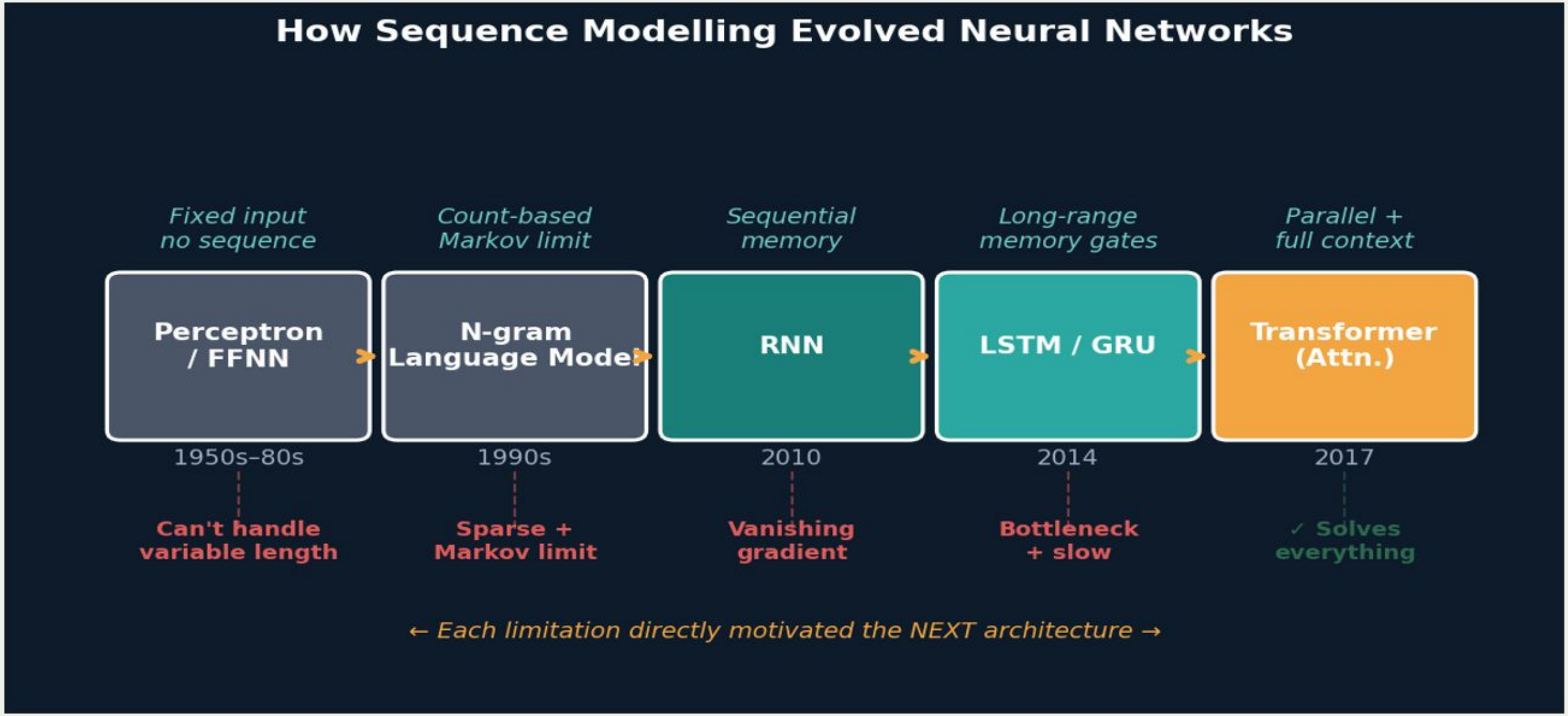
# Categories for Language Models

- Statistical Language Models
- Neural Language Models (NNLMs)
- Transformer-Based Language Models
- Large Language Models (LLMs)
- Multilingual & Low-Resource LMs

# Classical sequence architectures

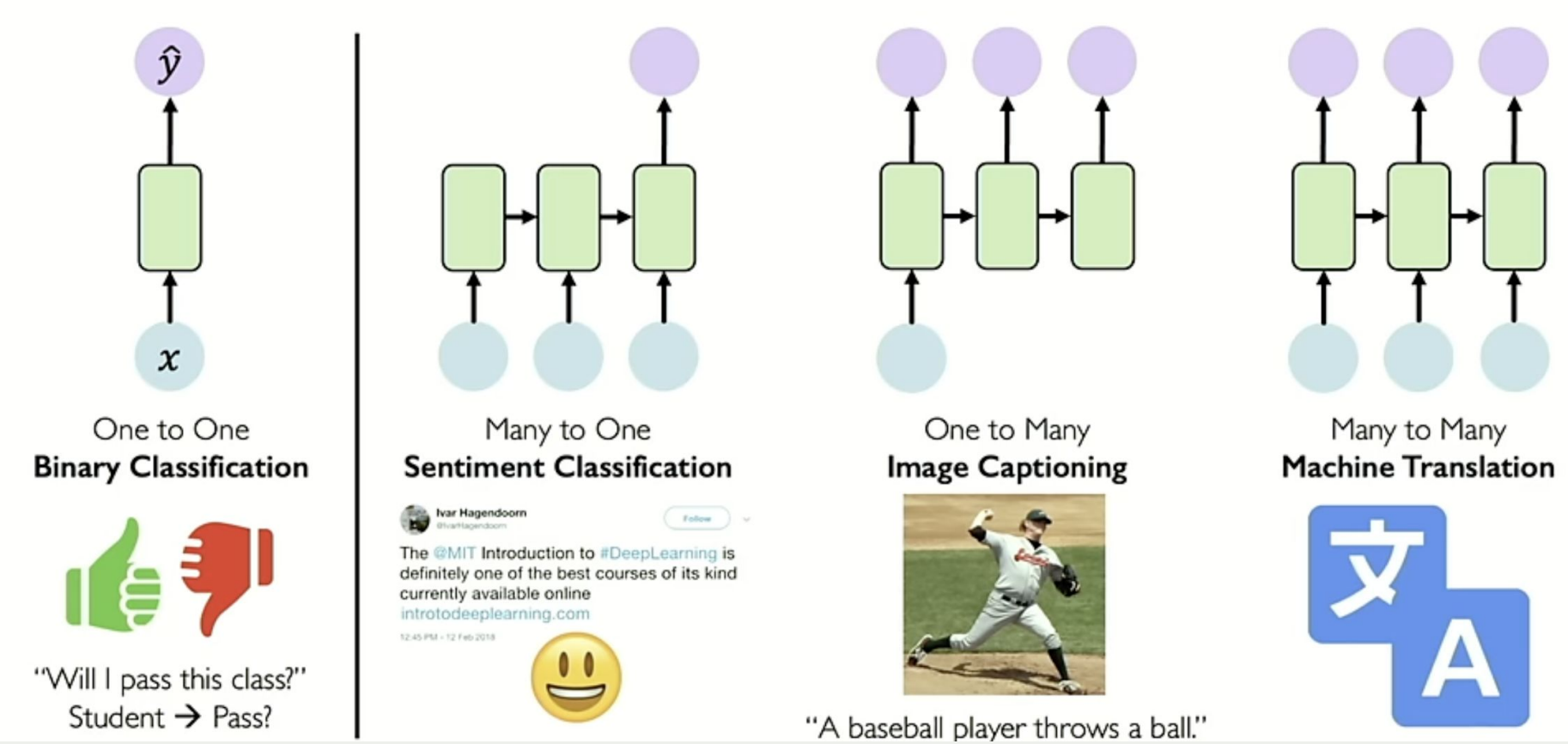
Classical sequence architectures:

- Recurrent Neural Networks (RNNs)
- Long Short-Term Memory Networks (LSTMs)
- Gated Recurrent Units (GRUs)
- Encoder–Decoder architectures (precursor to Transformers)




Every architecture limitation in handling sequences directly motivated the NEXT breakthrough — from perceptrons to Transformers.

# Sequence Modelling Application




All four panels are "sequence modeling" because the model is learning a conditional mapping between ordered data (a sequence) and an output that may be a single label or another sequence. A unifying view is:

# Key Challenges in Sequence Modelling

- **Variable Length: sentences range from 1 to 1000+ words — model must handle all**
- **Long-range dependencies**
- **Compositionality: meaning of a phrase ≠ sum of word meanings**
  - → Negation: 'not bad' ≠ 'not' + 'bad'
  - → Idioms: 'kick the bucket' is not about kicking
- Order sensitivity: word permutations change meaning completely

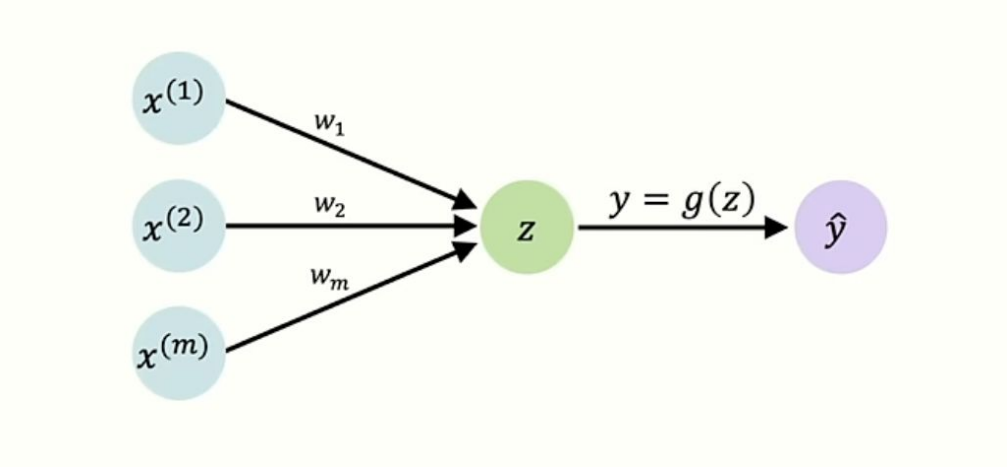
***These challenges motivate every architectural choice we'll discuss next***

# Neurons with Recurrence

# Perceptron Revisited

- The **Perceptron** is one of the earliest and simplest learning algorithms for binary classification.

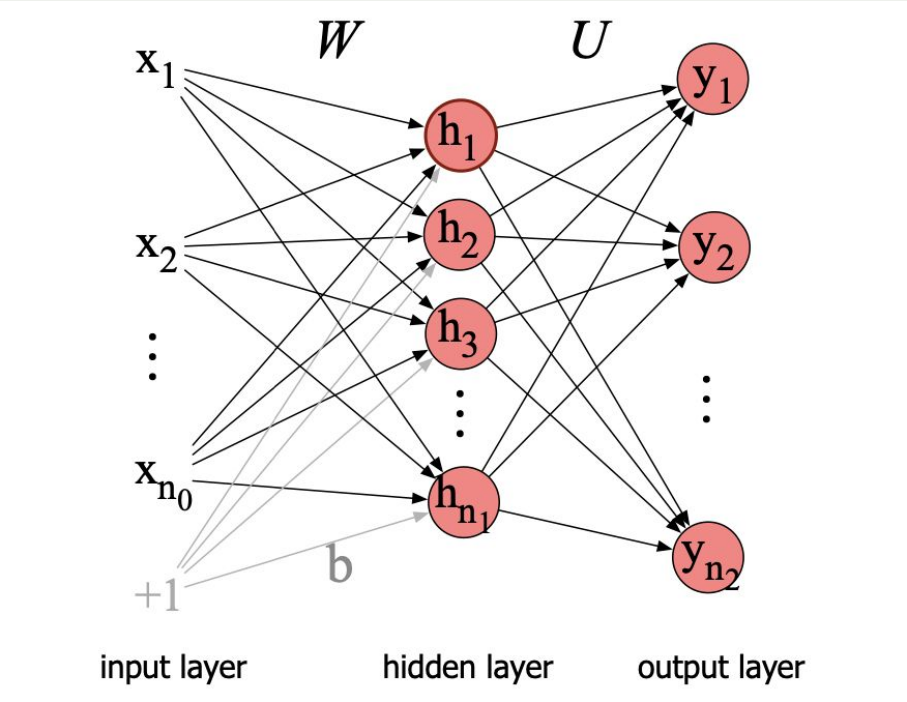
- It learns a linear decision boundary by repeatedly adjusting its weight vector based on mistakes.



# Feed-Forward Network

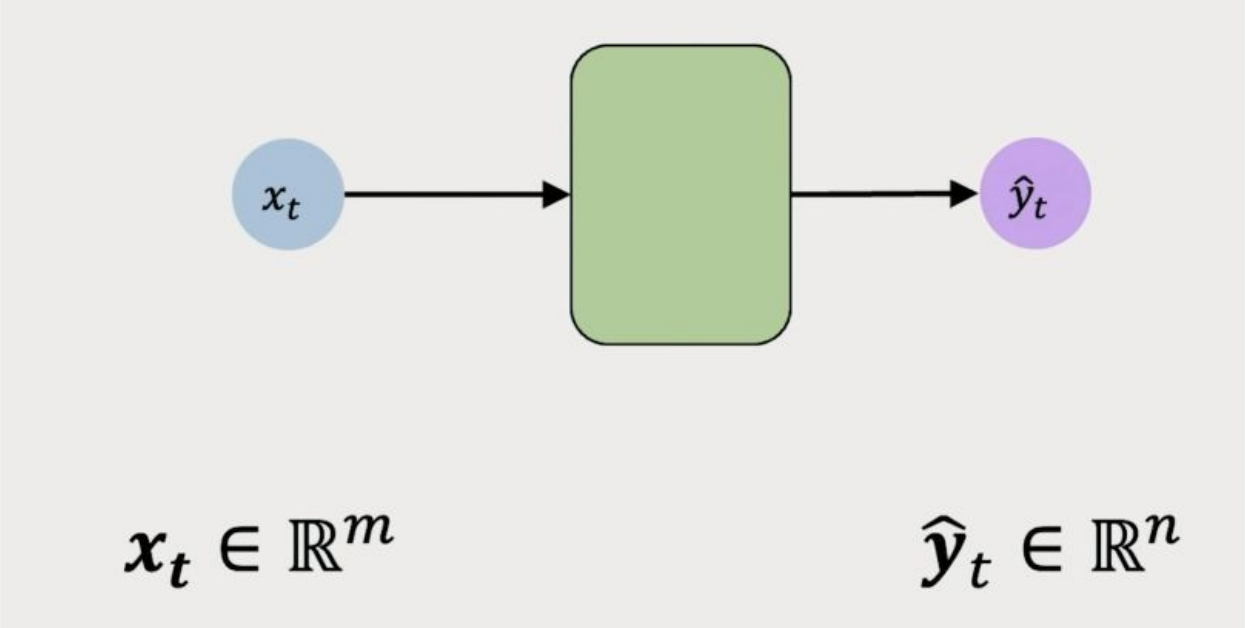
A **Feed-Forward Neural Network (FNN)**, also called a *multi-layer perceptron (MLP)*, is a basic neural architecture where information flows in one direction

**layers of perceptrons** → can model nonlinear decision boundaries and richer semantic relationships.



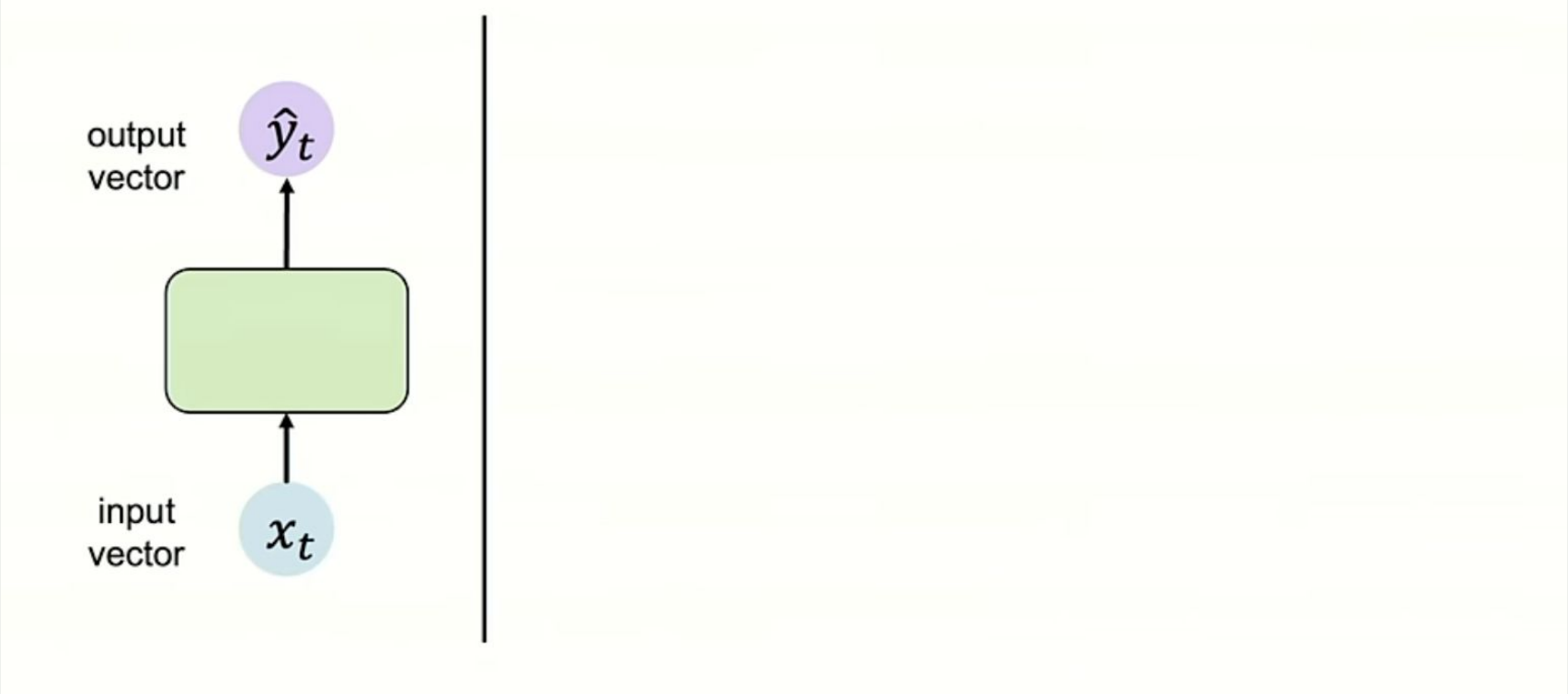

**No notion of time stap of sequence**

# Feed-Forward Network Revisited

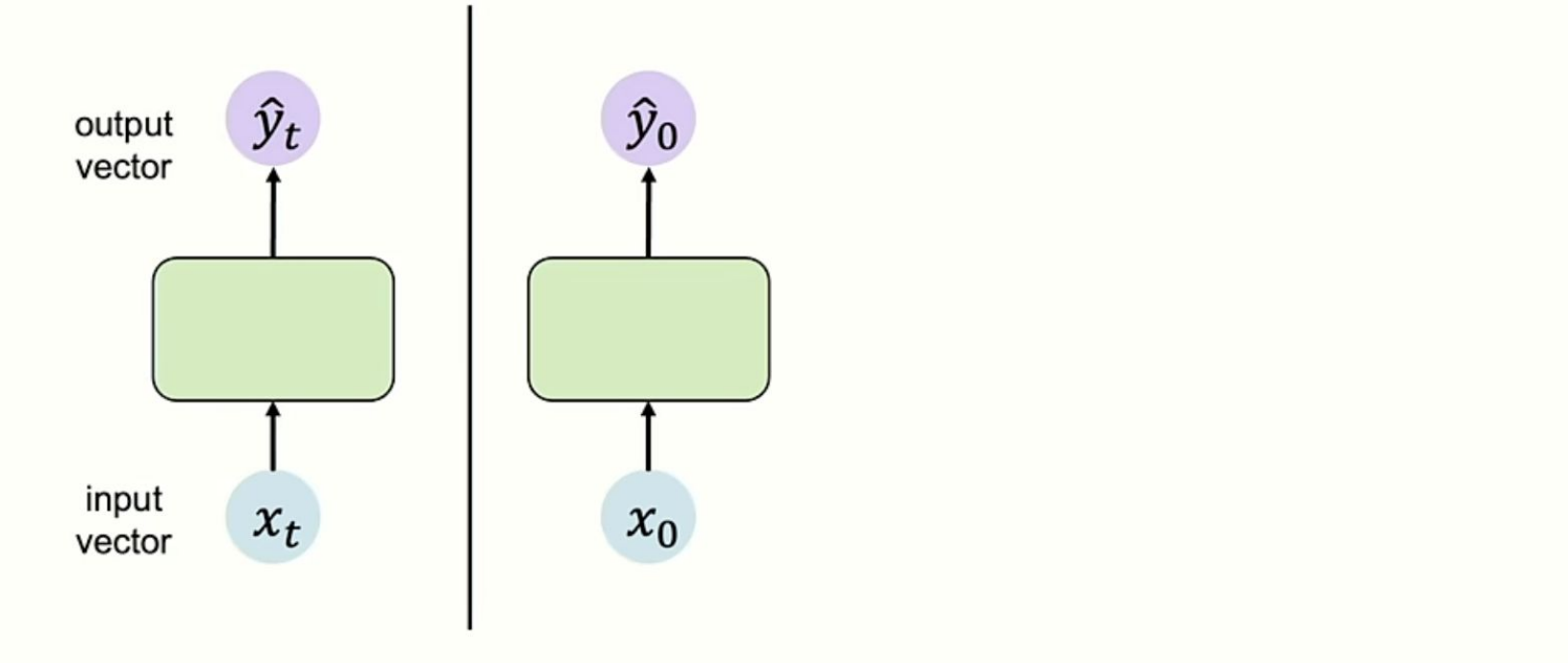



- Still no notion don't handle sequence data
- How can we handle sequence data?

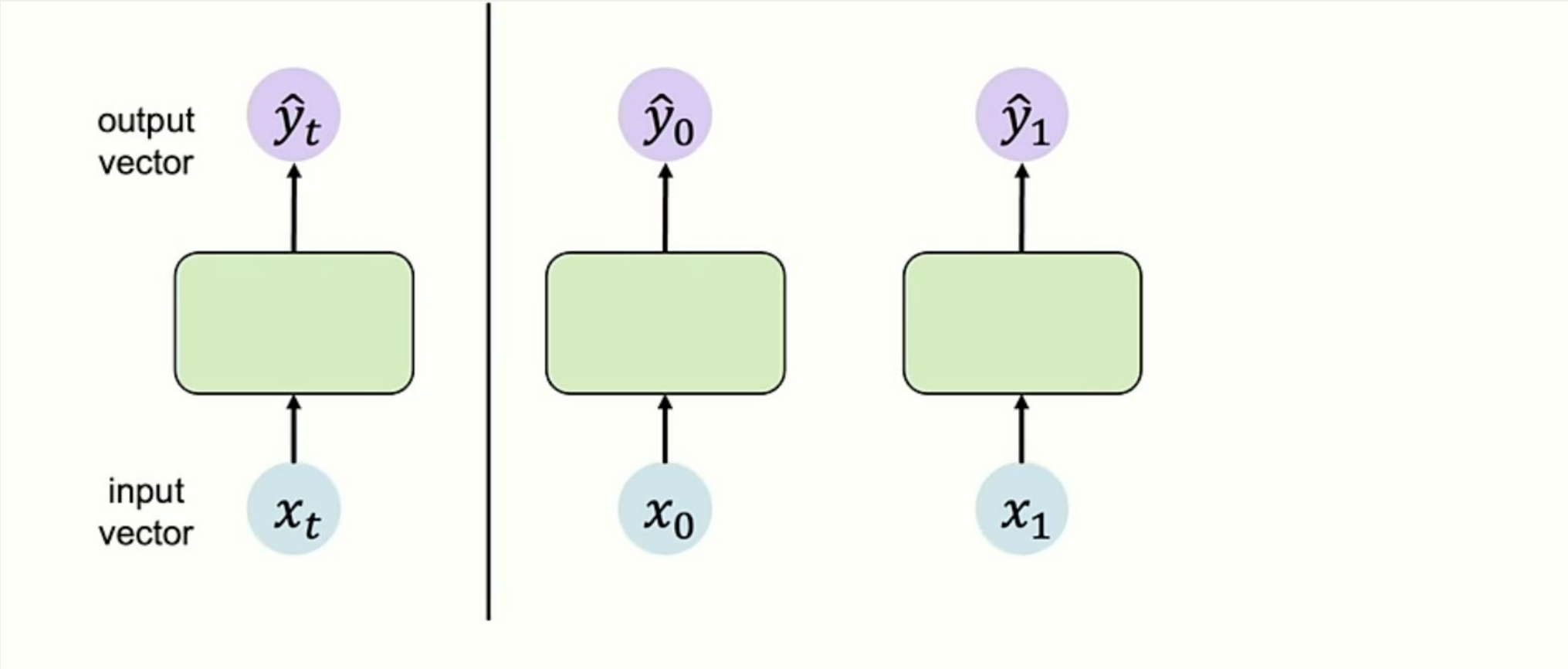
# Handling Individual Time-Step



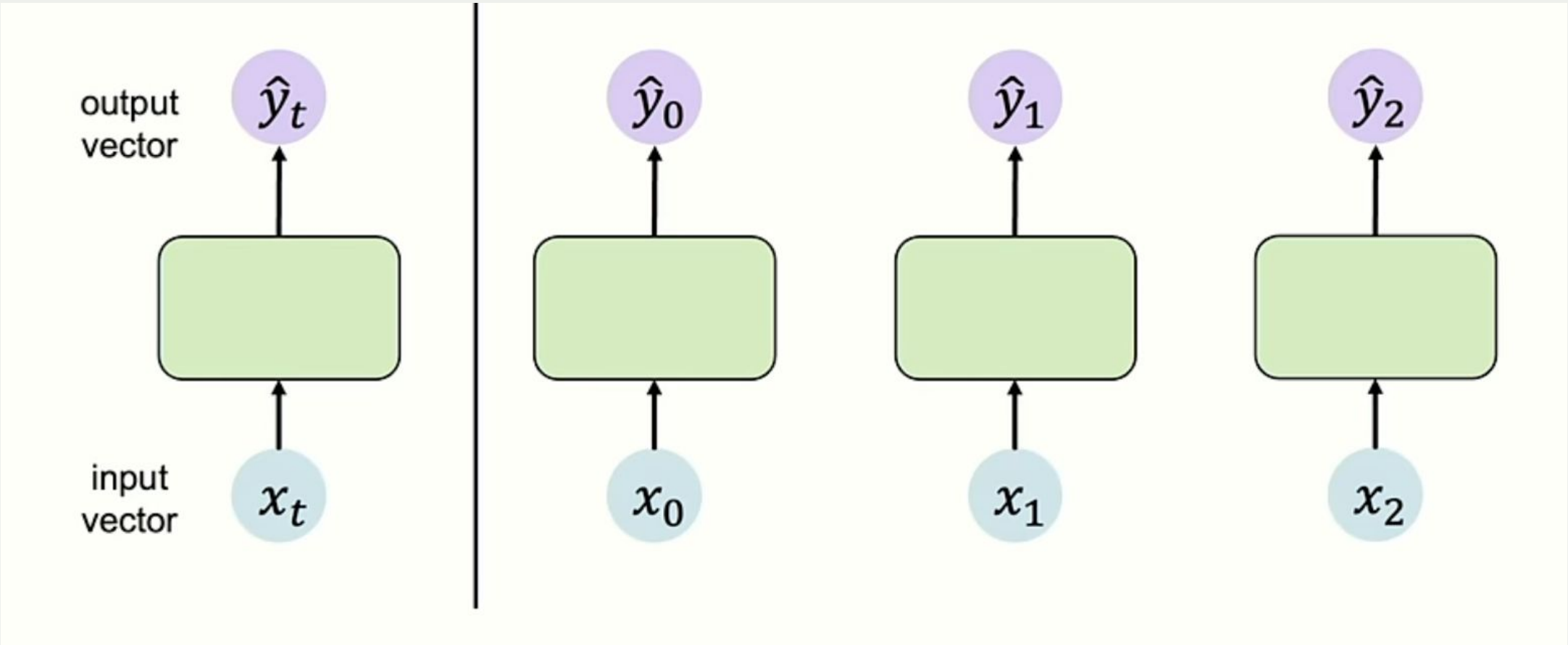
# Handling Individual Time-Step



# Handling Individual Time-Step

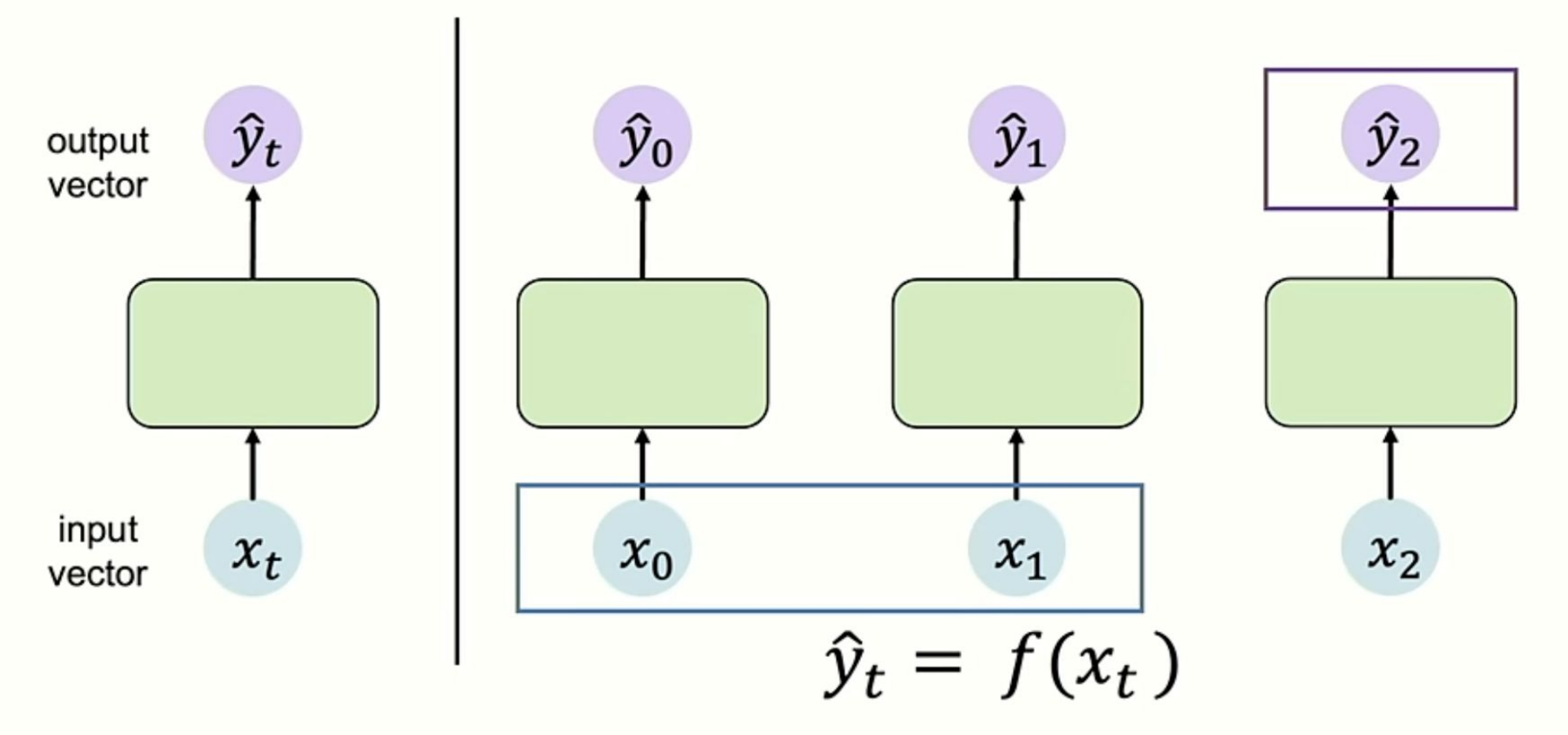


# Handling Individual Time-Step



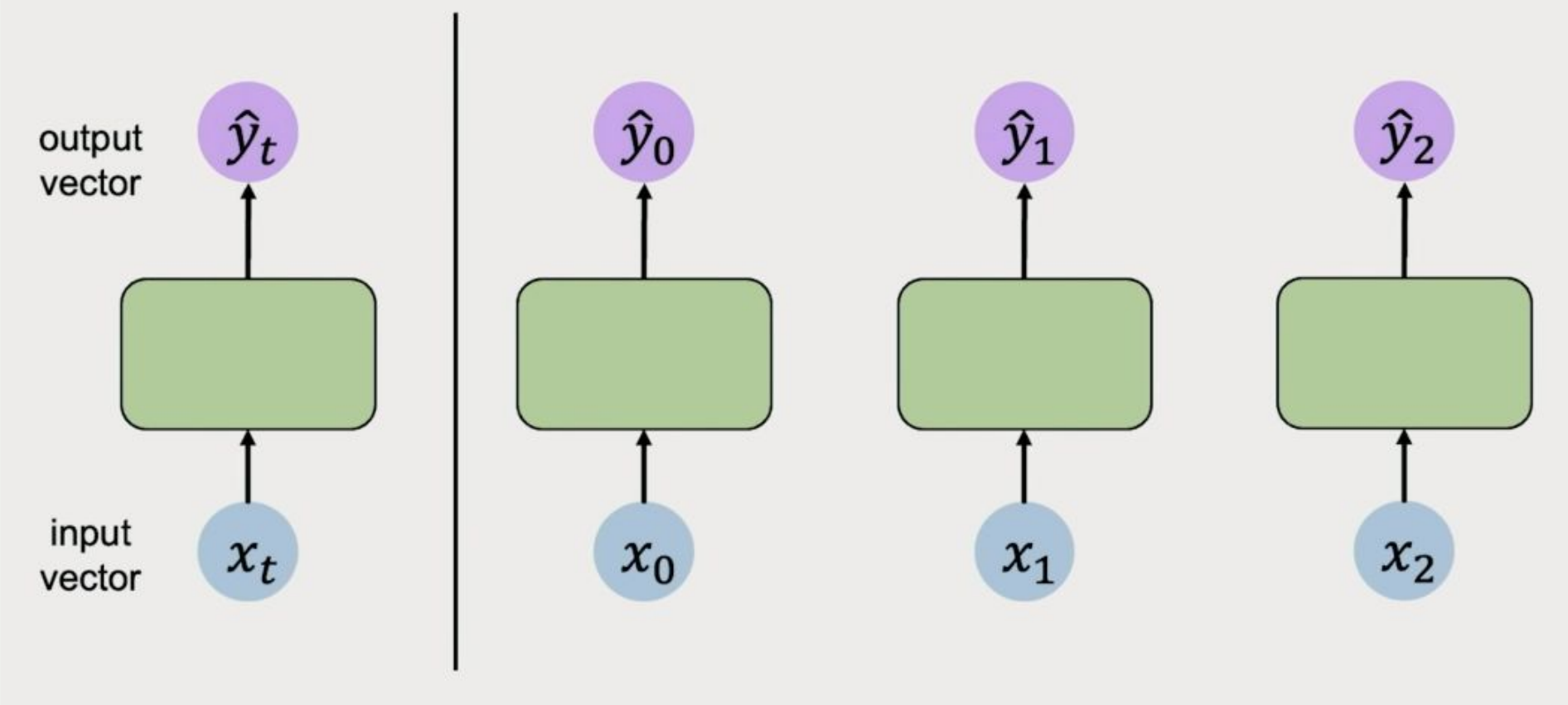

no notion of handling sequence data. How can we handle sequence data?

# Handling Individual Time-Step



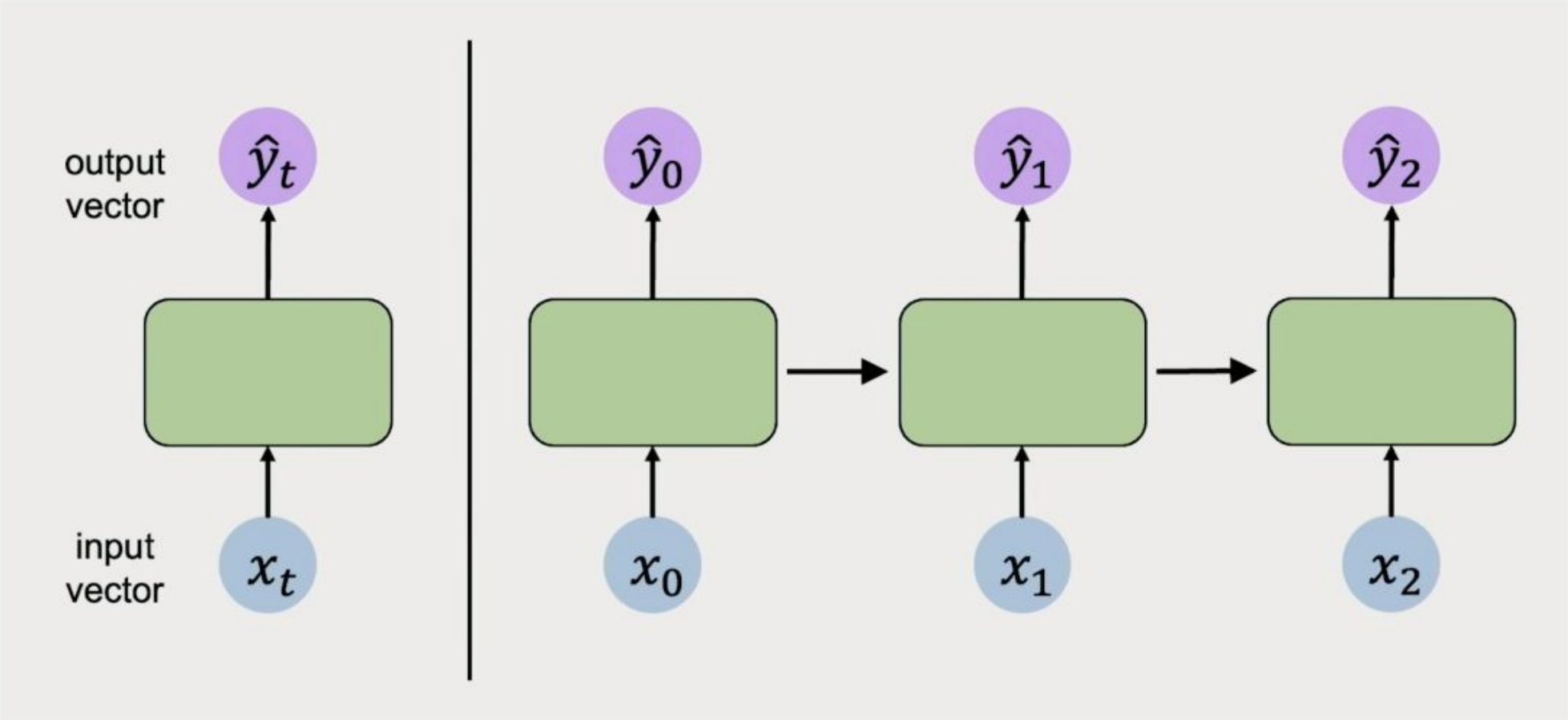

no notion of handling sequence data. How can we handle sequence data?

# Neurons with Recurrent



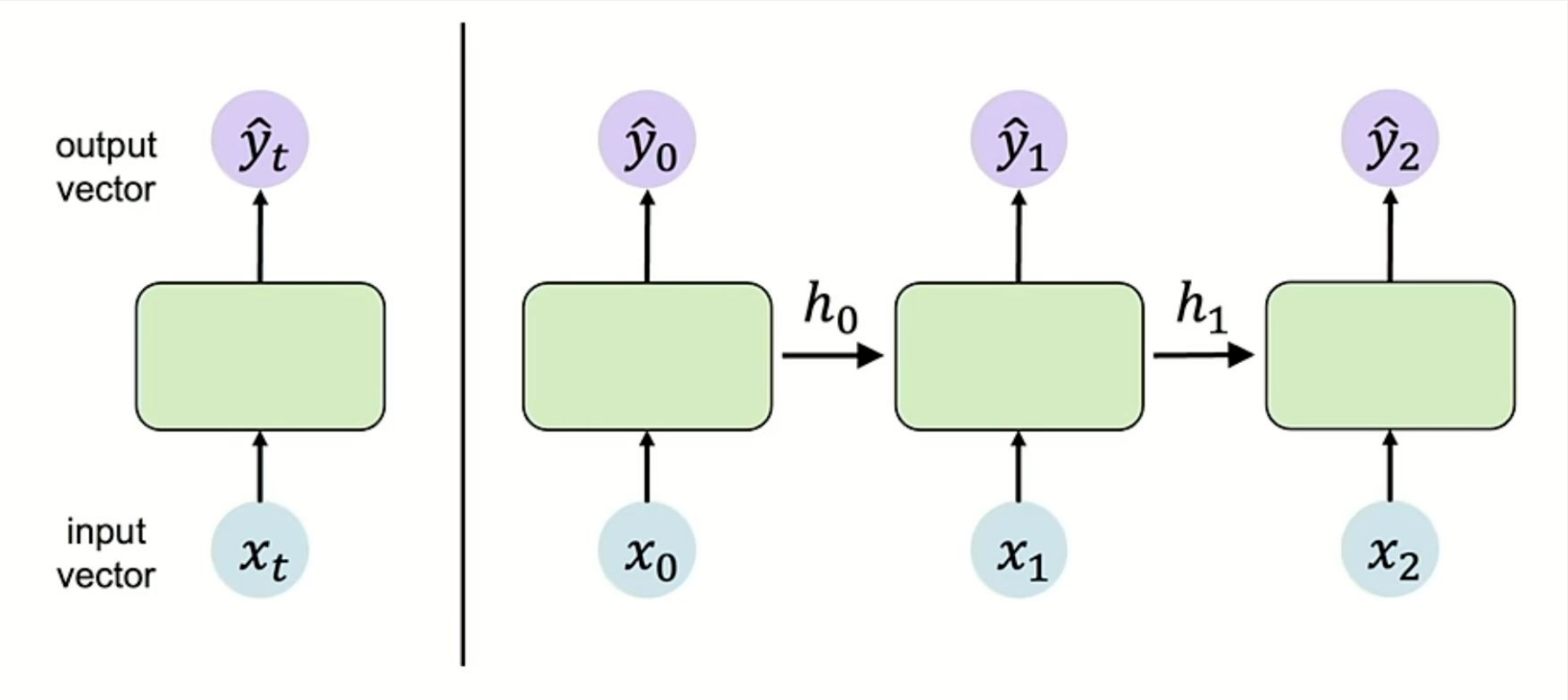

How can we handle sequence data? Recurrent?

# Neurons with Recurrent

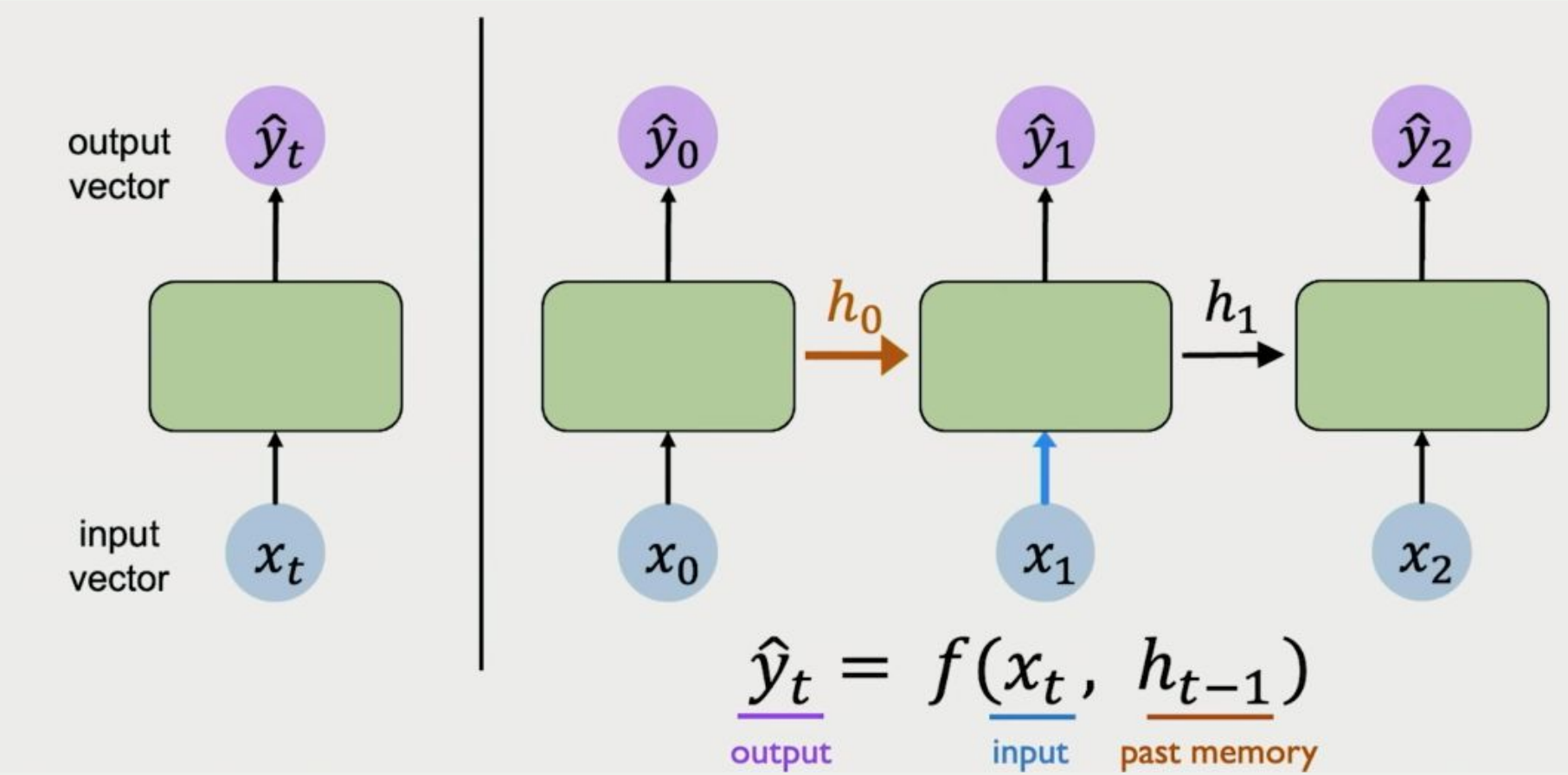



How can we handle sequence data? Recurrent?

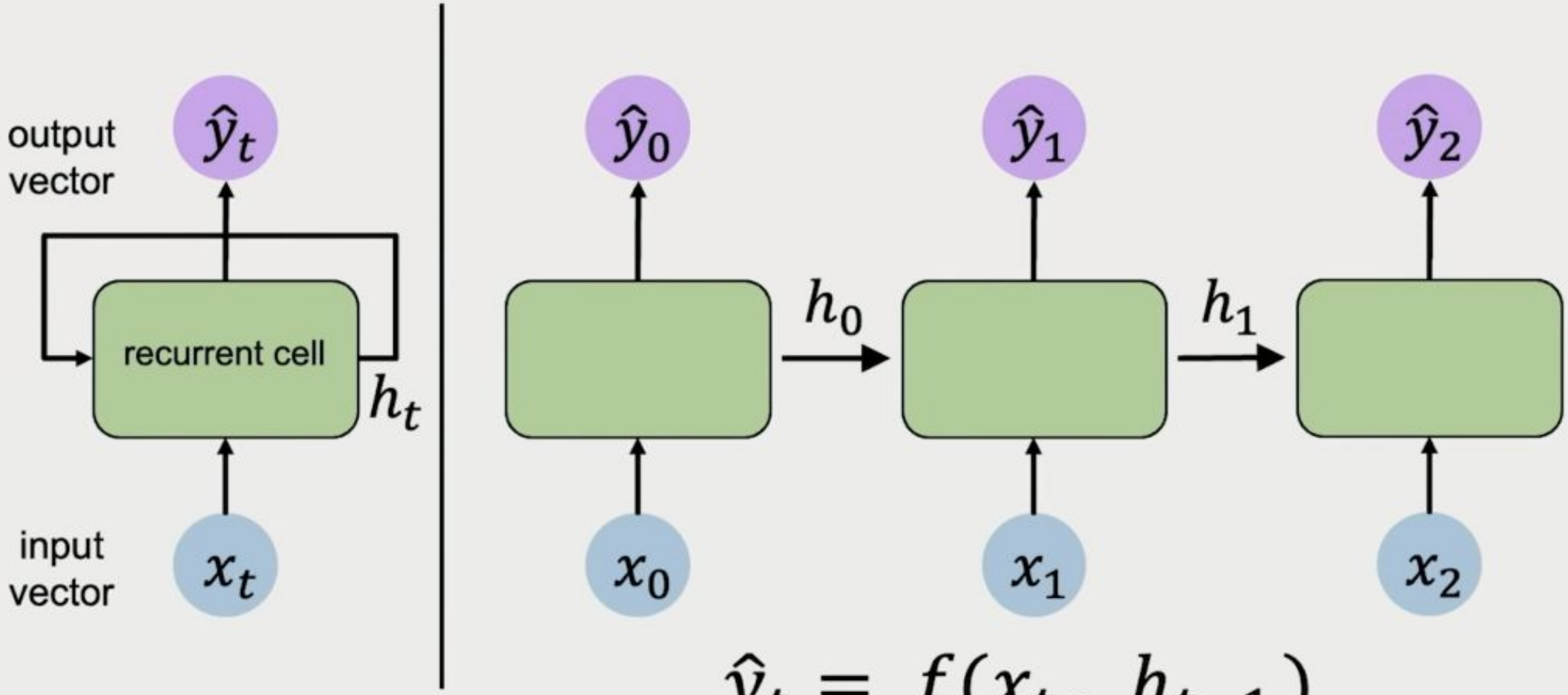
# Neurons with Recurrent



# Neurons with Recurrent



# Neurons with Recurrent




$$\underbrace{\hat{y}_t}_{\text{output}} = f(\underbrace{x_t}_{\text{input}}, \underbrace{h_{t-1}}_{\text{past memory}})$$

# Recurrent

The term "recurrent" generally means occurring repeatedly or in cycles, and in the context of Recurrent Neural Networks (RNNs), it specifically refers to the mechanism of repeatedly applying the same computational process to sequential data.

The recurrence enables the network to "remember" previous states over time, giving RNNs the ability to capture patterns and dependencies across sequential data.

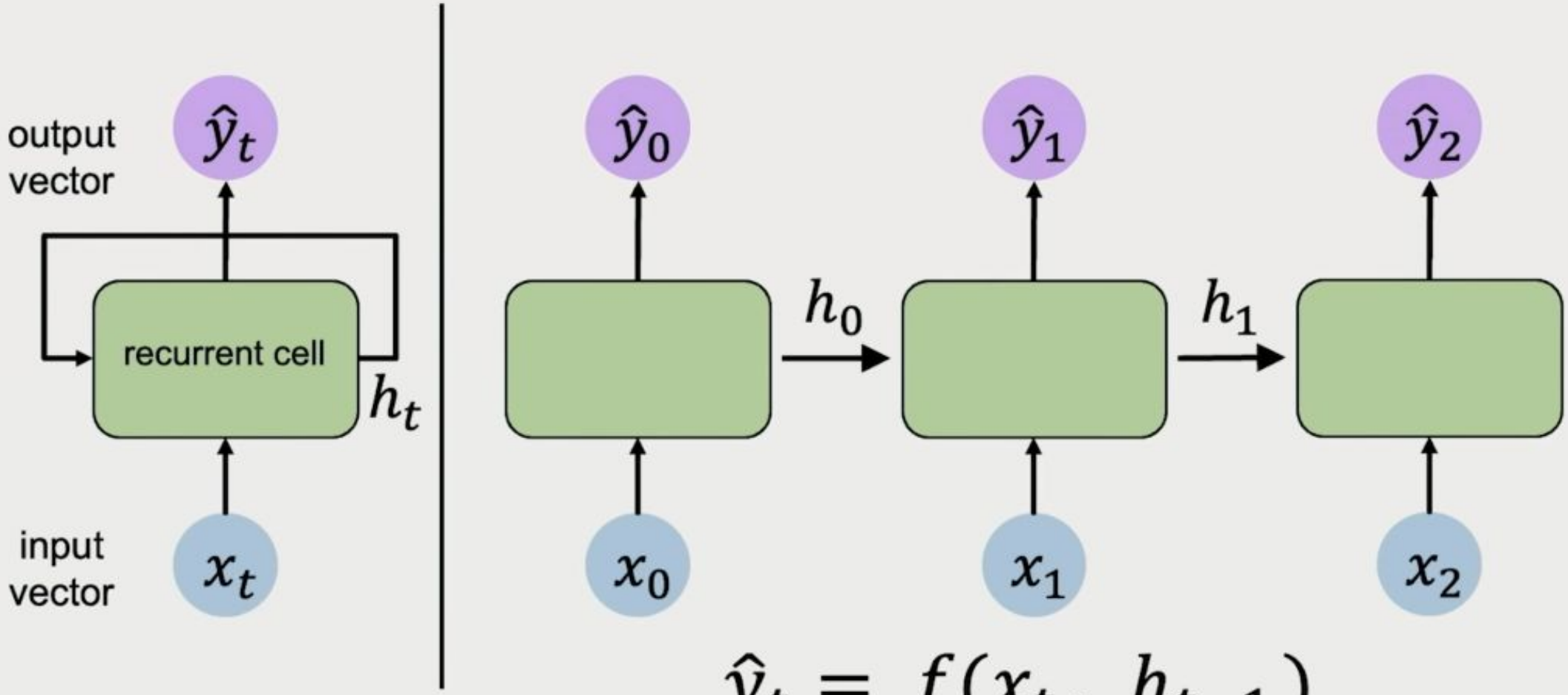
# Recurrent

The recurrence is foundational concept sequence modelling called Recurrent Neural Networks (RNN)

- **Text sequences** (e.g., sentences in natural language processing),
- **Time-series data** (e.g., stock prices, sensor readings),
- **Speech data** (e.g., audio signals),
- **Video frames** (e.g., in action recognition).

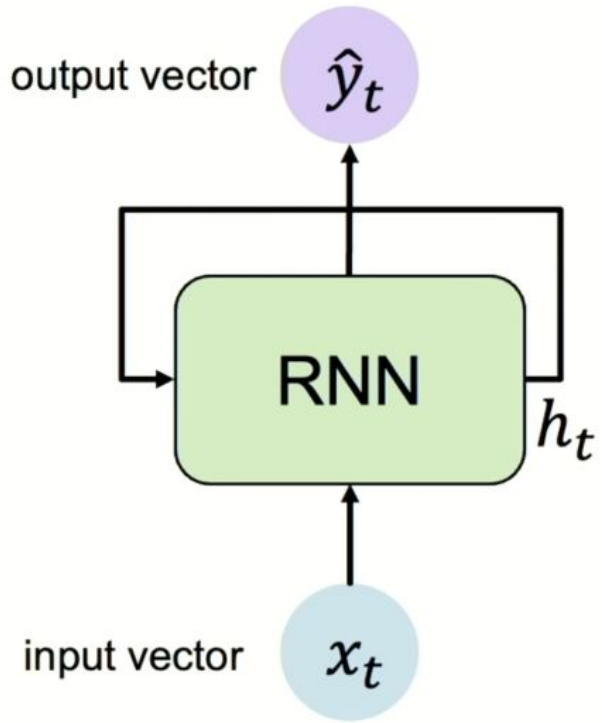
# Recurrent Neural Networks (RNNs)

# Neurons with Recurrent



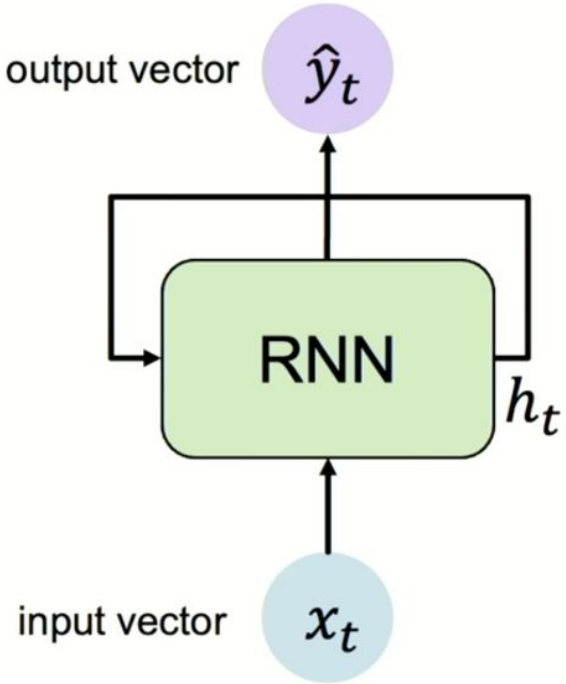

$$\underbrace{\hat{y}_t}_{\text{output}} = f(\underbrace{x_t}_{\text{input}}, \underbrace{h_{t-1}}_{\text{past memory}})$$

# Recurrent Neural Network




RNNs have a **state**, $h_t$, that is updated **at each time step** as a sequence is processed

# Recurrent Neural Network



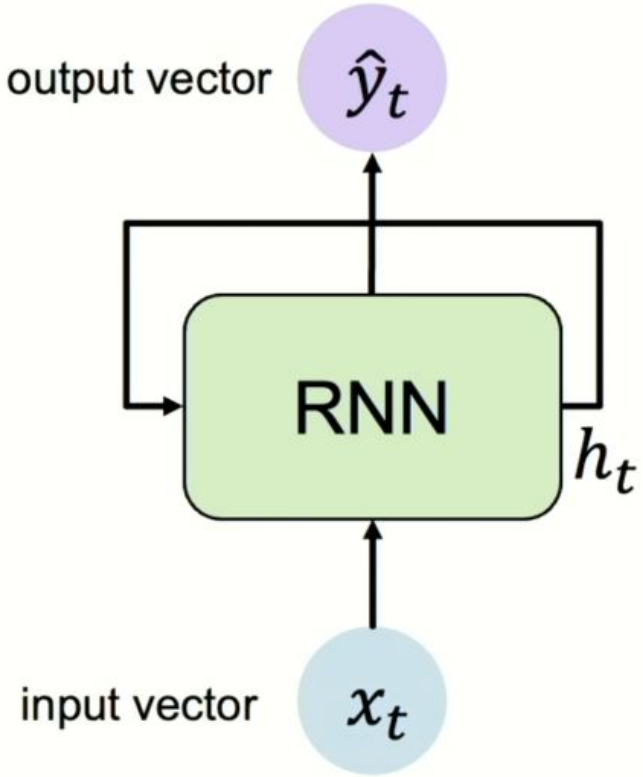

Apply a **recurrence relation** at every time step to process a sequence:

$$\boxed{h_t} = f_W(x_t, h_{t-1})$$

cell state

RNNs have a **state**, $h_t$, that is updated **at each time step** as a sequence is processed

# Recurrent Neural Network




Apply a **recurrence relation** at every time step to process a sequence:

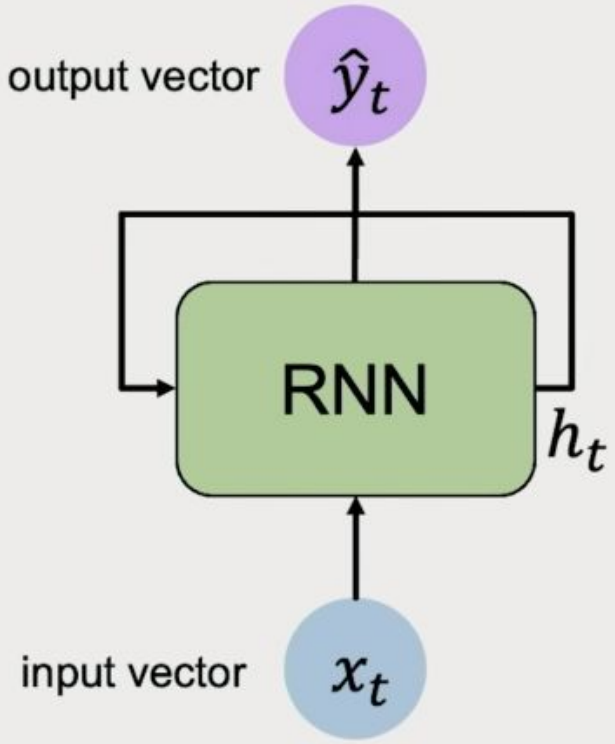
$$\boxed{h_t} = \boxed{f_W}(\boxed{x_t}, \boxed{h_{t-1}})$$

cell state — function with weights W — input — old state

**Note: the same function and set of parameters are used at every time step**

RNNs have a **state**, $h_t$, that is updated **at each time step** as a sequence is processed

# RNN State Update and Output



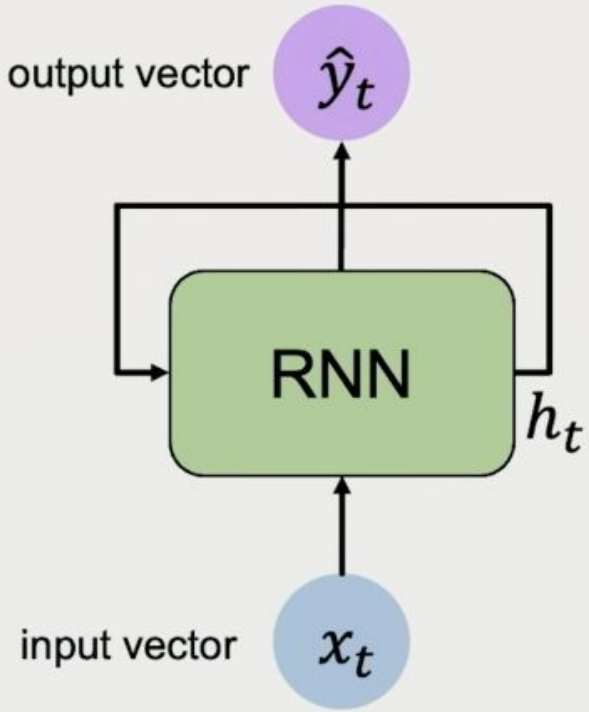

Update Hidden State

$$h_t = \tanh(\boldsymbol{W}_{\boldsymbol{hh}}^{\boldsymbol{T}} h_{t-1} + \boldsymbol{W}_{\boldsymbol{xh}}^{\boldsymbol{T}} x_t)$$

Input Vector

$x_t$

# RNN: Computational Graph Across Time




Output Vector

$$\hat{y}_t = \boldsymbol{W}_{\boldsymbol{hy}}^{\boldsymbol{T}} h_t$$

Update Hidden State

$$h_t = \tanh(\boldsymbol{W}_{\boldsymbol{hh}}^{\boldsymbol{T}} h_{t-1} + \boldsymbol{W}_{\boldsymbol{xh}}^{\boldsymbol{T}} x_t)$$

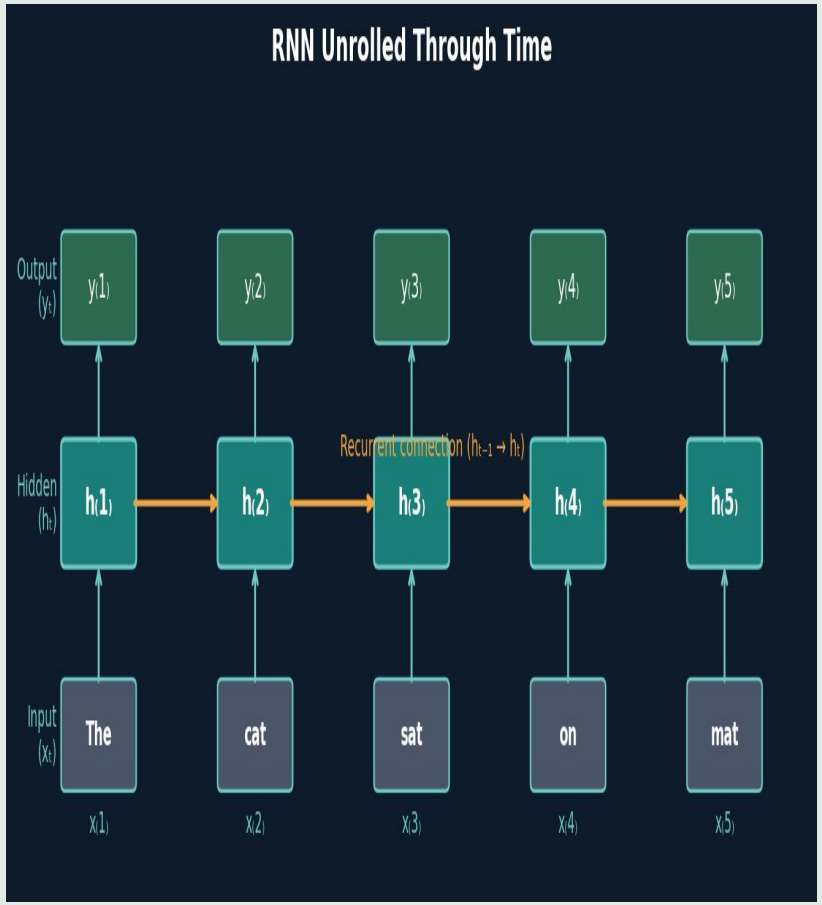
Input Vector

$$x_t$$

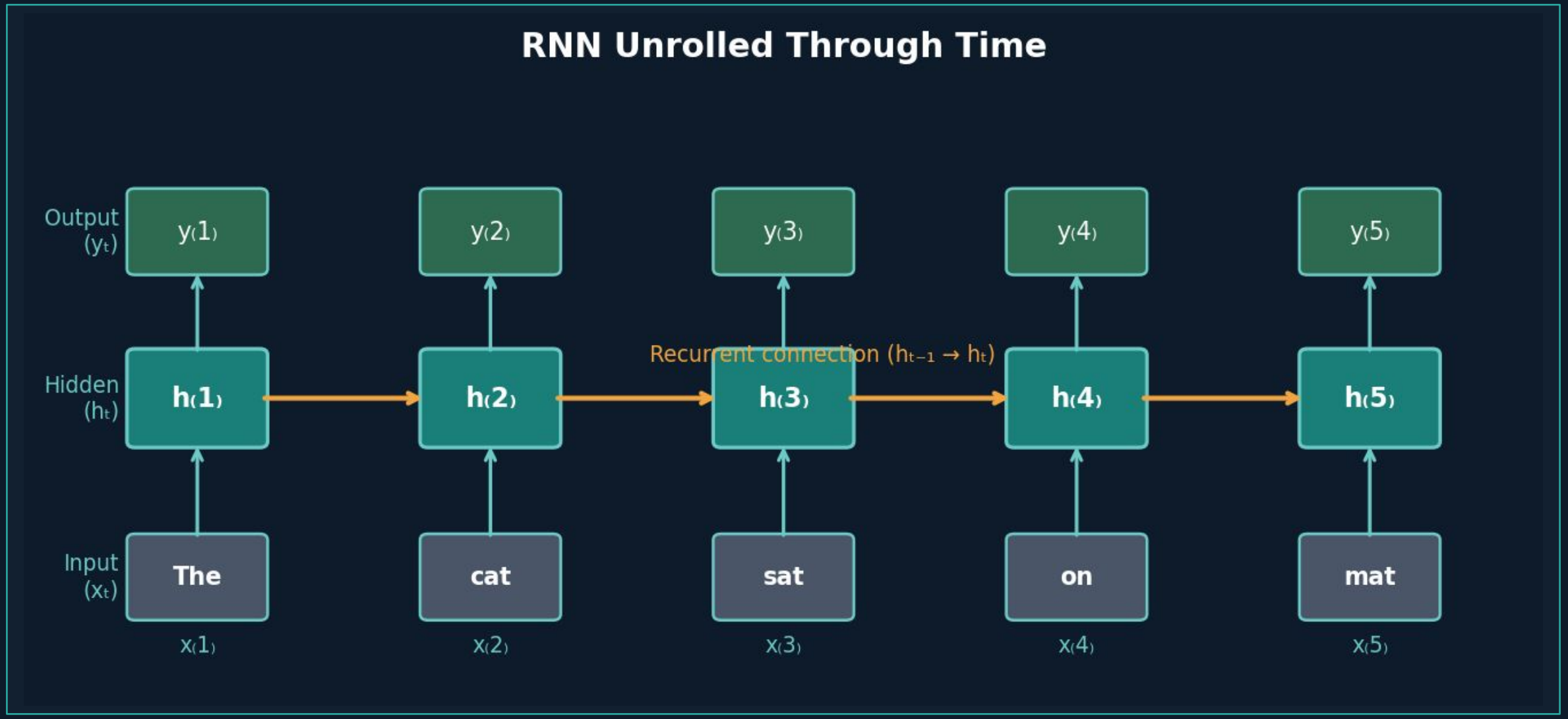
# Motivation for Recurrent Networks

- Humans process language word-by-word, maintaining a mental 'context'
- When you read 'the cat sat on the…', you predict 'mat' — not randomly, but from memory
- **RNNs model this: a hidden state $h_t$ carries information from position 1 through t**
- At each step, the model reads the new word AND the accumulated context
- The same set of parameters is reused at every time step — weight sharing over time
- ***Core recurrence:*** $h_t = f(W_x \cdot x_t + W_h \cdot h_{t-1} + b)$
- $h_0$ is typically initialised to zero — the 'blank memory' at the start

*The hidden state $h_t$ is the heart of sequence modelling*



# RNN Unrolled Through Time

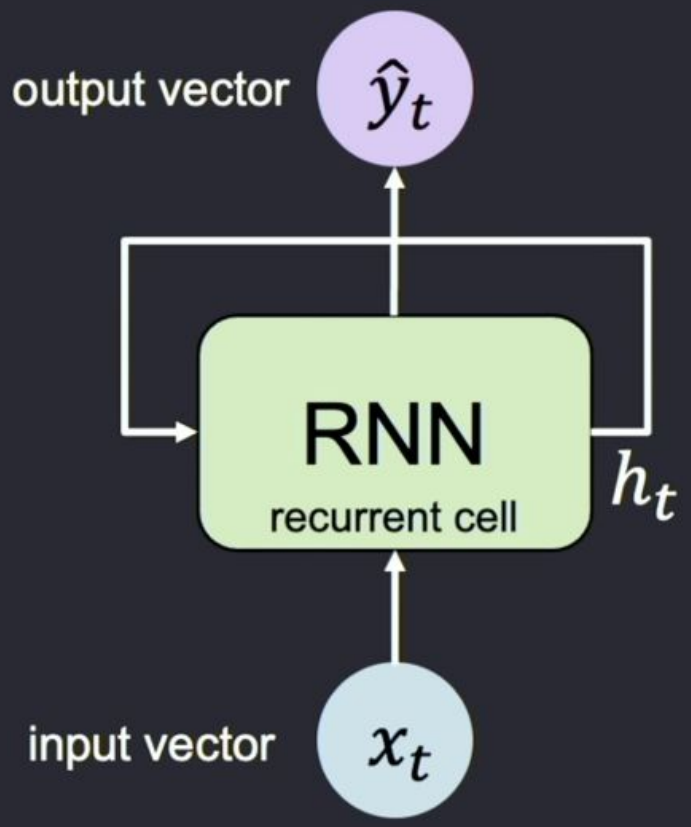



Imperial College London

*Same $W_x$, $W_\square$, $W_Y$ used at every timestep — weight sharing over time. Gold arrows = recurrent connections*

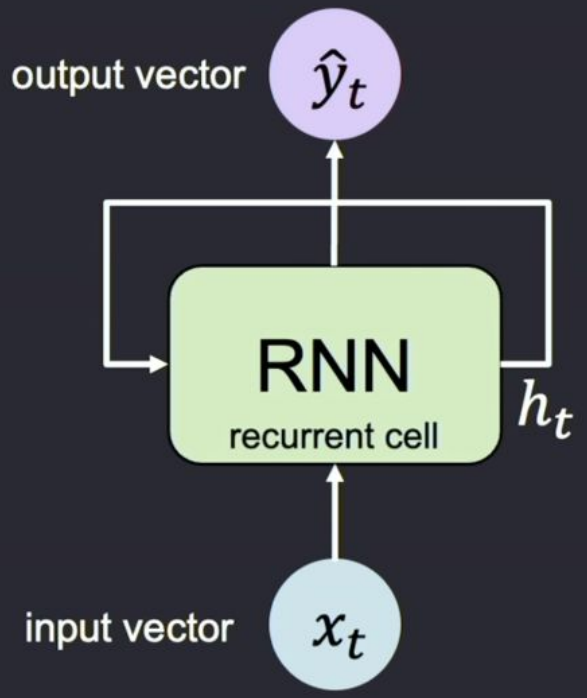
# RNN Intuition: Pseudocode

```
my_rnn = RNN()
hidden_state = [0, 0, 0, 0]

sentence = ["I", "love", "recurrent", "neural"]

for word in sentence:
    prediction, hidden_state = my_rnn(word, hidden_state)

next_word_prediction = prediction
# >>> "networks!"
```



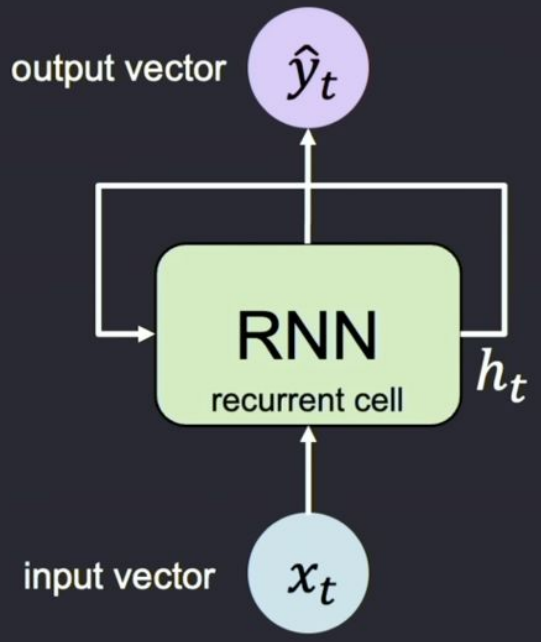
# RNN Intuition: Pseudocode

```
my_rnn = RNN()
hidden_state = [0, 0, 0, 0]

sentence = ["I", "love", "recurrent", "neural"]

for word in sentence:
    prediction, hidden_state = my_rnn(word, hidden_state)

next_word_prediction = prediction
# >>> "networks!"
```



# RNN Intuition: Pseudocode

```
my_rnn = RNN()
hidden_state = [0, 0, 0, 0]

sentence = ["I", "love", "recurrent", "neural"]

for word in sentence:
    prediction, hidden_state = my_rnn(word, hidden_state)

next_word_prediction = prediction
# >>> "networks!"
```



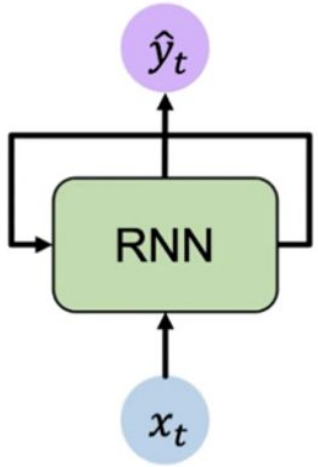
# RNN Forward Pass: Step by Step

```
# Vanilla RNN equations
# x□  : input word embedding at time t
# h□  : hidden state (memory) at time t

# Parameters (shared across ALL time steps)
Wx = weight_matrix(embed_dim, hidden_dim)   # input weights
Wh = weight_matrix(hidden_dim, hidden_dim)  # recurrent weights
Wy = weight_matrix(hidden_dim, vocab_size)  # output weights

h0 = zeros(hidden_dim)   # initial hidden state

for t in range(seq_len):
    ht = tanh( Wx @ xt  +  Wh @ h_prev  +  bias )
    yt = softmax( Wy @ ht + output_bias )   # next word probability
    h_prev = ht
```

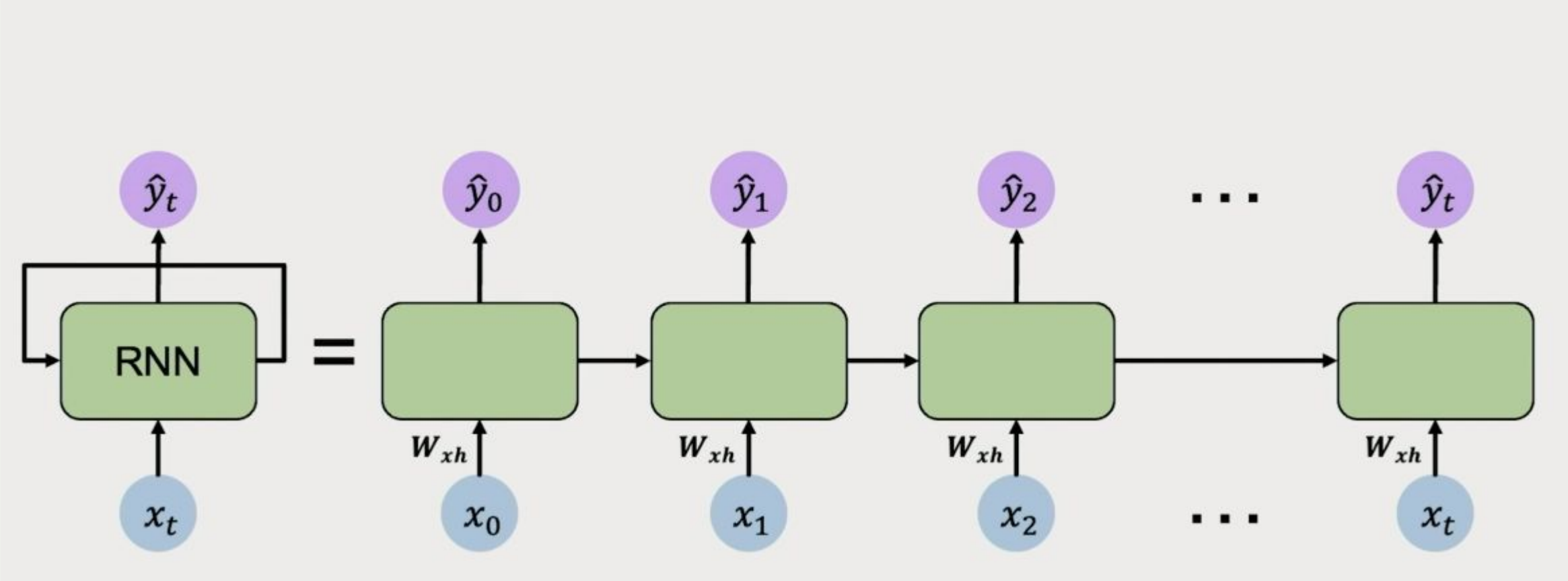
*Same Wx, Wh, Wy used at every timestep — weight sharing over time*

Imperial College London

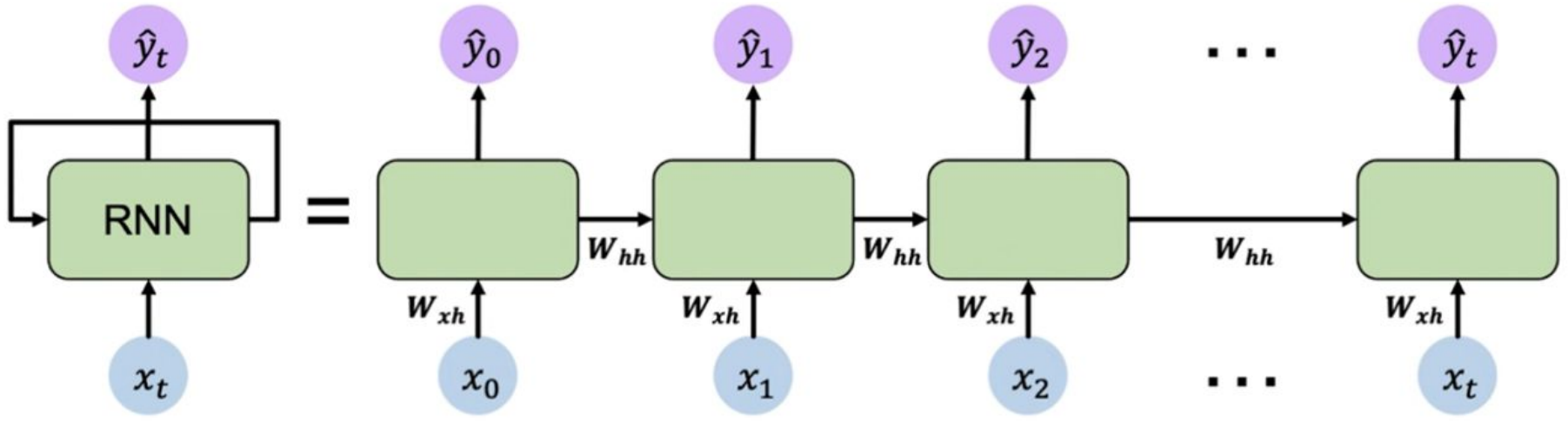
# RNN: Computational Graph Across Time



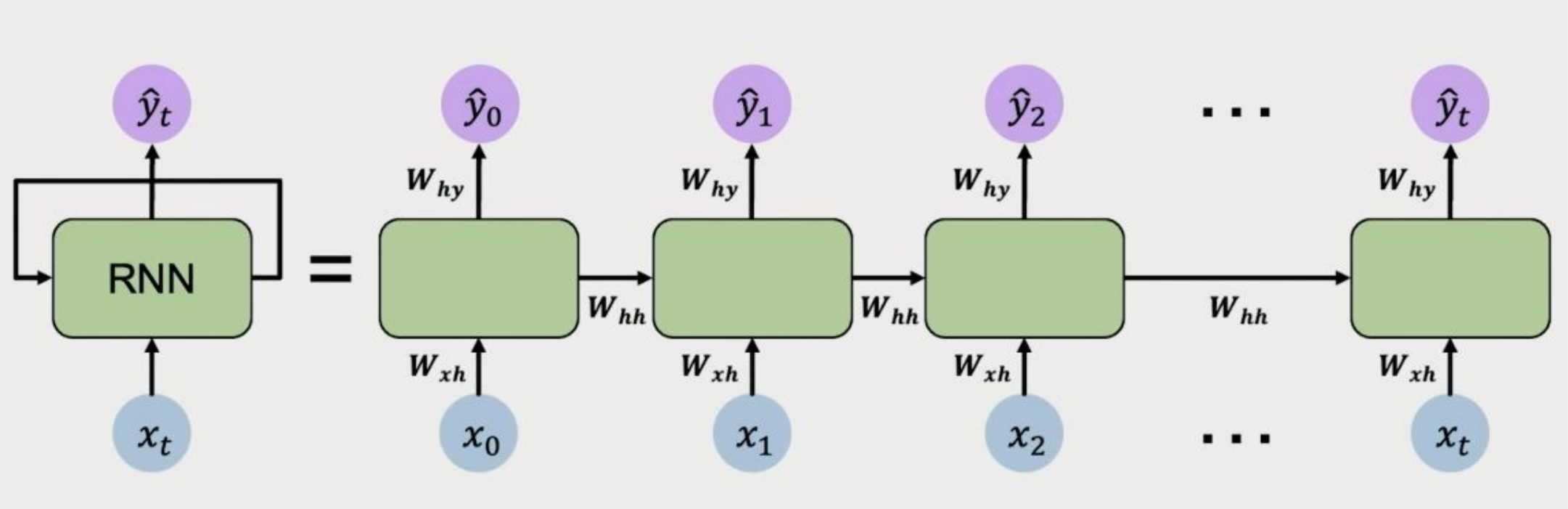
# RNN: Computational Graph Across Time



# RNN: Computational Graph Across Time

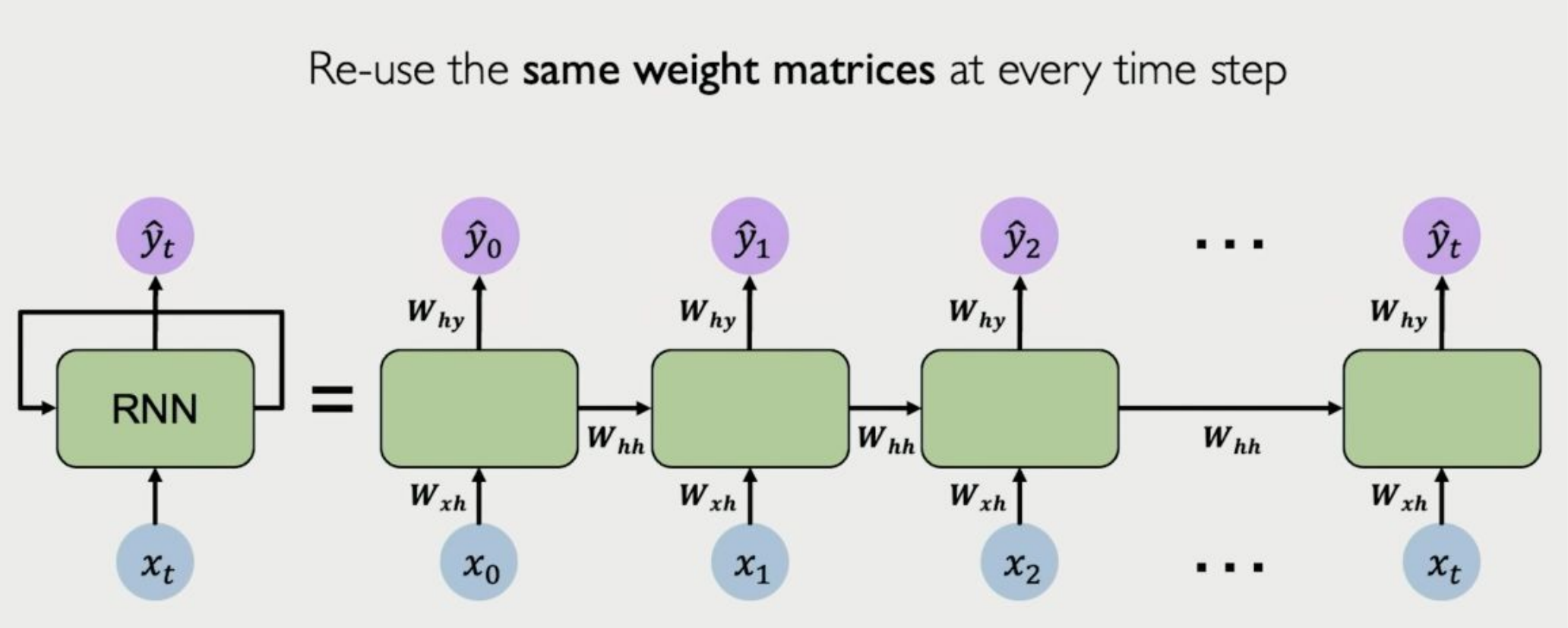


# RNN: Computational Graph Across Time

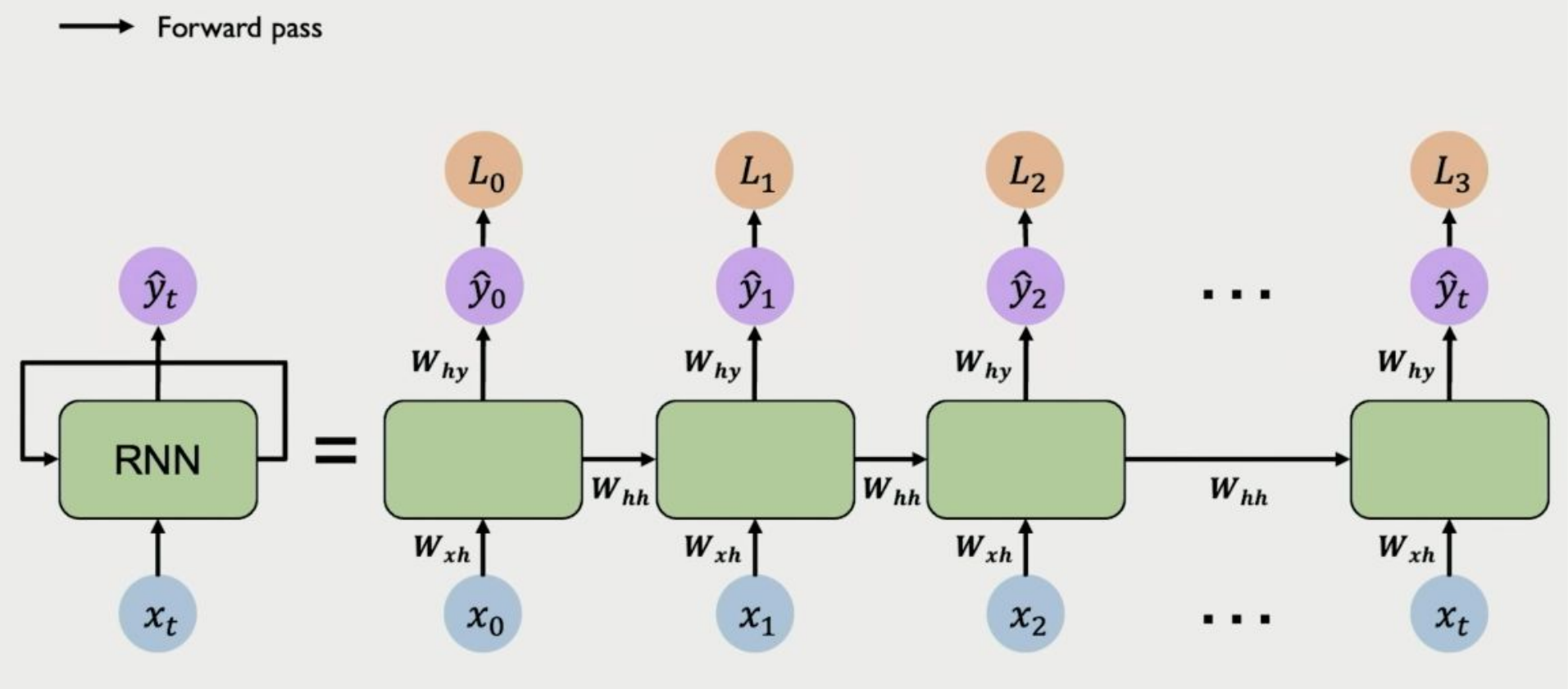



Reuse the same weight at every time step

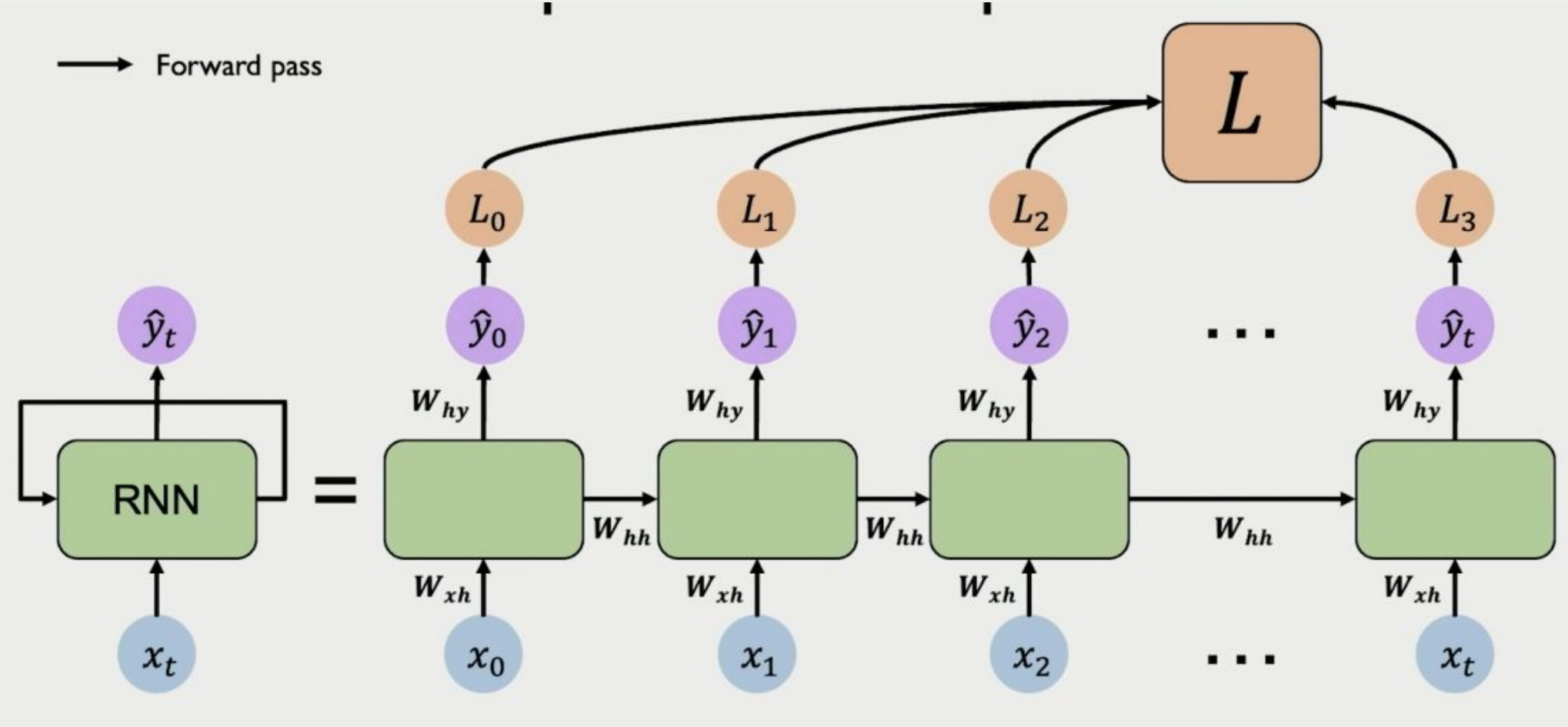
# RNN: Computational Graph Across Time



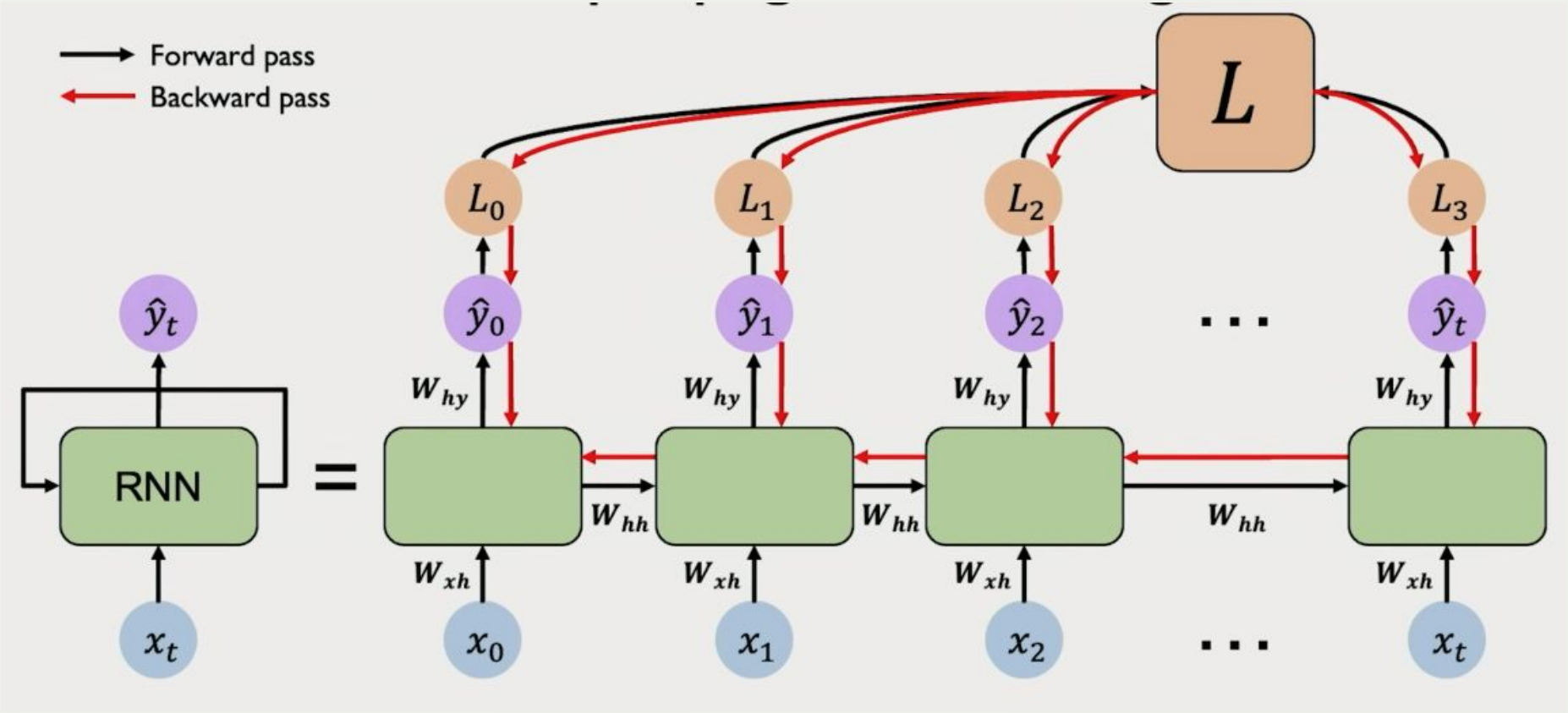
# RNN: Computational Graph Across Time



# RNN: Computational Graph Across Time



# Recall: Backpropagation in Feed Forward



# Problems with RNNs

**Problem**

- Vanishing Gradient
- Exploding Gradient
- Difficulty with Long-Term Memory
- Slow Training
- Limited Parallelization

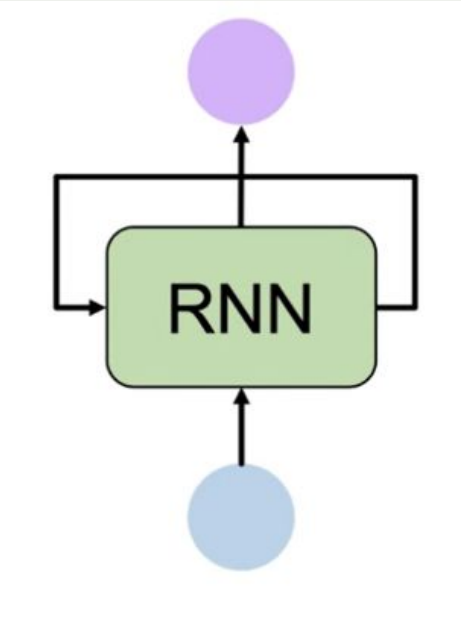
# Common mitigations

- **Vanishing/long-term memory:** gated RNNs (LSTM/GRU), better initialization, residual connections, normalization, attention.
- **Exploding gradient:** gradient clipping, smaller learning rates, careful initialization.
- **Slow training / limited parallelization**: Transformers (more parallel)

# Sequence Modelling: Design Criteria

To model sequences, we need to:

- Handle variable-length sequences
- Track long-term dependencies
- Maintain information about order
- Share parameters across the sequence




**Recurrent Neural Networks (RNNs)** meet these sequence modeling design criteria.

# Handle variable-length sequences

The food was great

vs.

We visited a restaurant for lunch

vs.

We were hungry but cleaned the house before eating

# Track long-term dependencies

"**France** is where I grew up, but I now live in Boston. I speak fluent ____."

We need information from **the distant past** to accurately predict the correct word.

# Capture Differences in Sequence Order

The food was good, not bad at all.

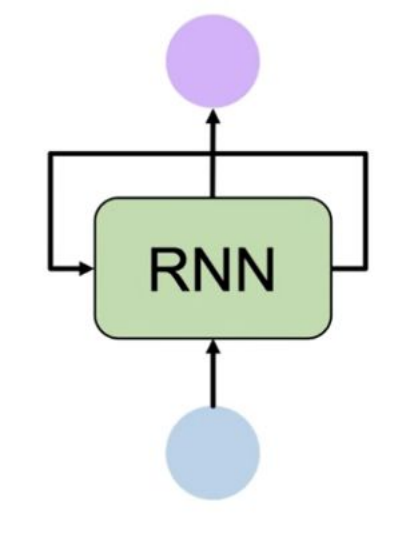
vs.

The food was bad, not good at all.

# Sequence Modelling: Design Criteria

To model sequences, we need to:

- **Handle variable-length sequences**
- **Track long-term dependencies**
- **Maintain information about order**




**Recurrent Neural Networks (RNNs)** meet these sequence modelling design criteria.

# Practice: RNN from Scratch

- CNN for text classification
- RNN for text classification

# Q and A